

eda_life_style_data_v2.0

December 1, 2025

1 EDA V2 – Life_Style_Data.csv

1.1 Analyse exploratoire avec X et Y finalisés

Ce notebook réalise l'exploration du dataset **Life_Style_Data.csv** en se basant sur le schéma validé pour l'entraînement du modèle prédictif.

- **Target (y) :** `Experience_Level`
- **Features (X) :**
 - `Age`
 - `Gender`
 - `Weight (kg)`
 - `Height (m)`
 - `Workout_Type`
 - `Workout_Frequency (days/week)`

L'objectif est de vérifier la structure, la qualité et la distribution de ces variables, afin de garantir que le dataset est prêt pour la modélisation supervisée.

1.2 Chargement et inspection initiale

1.2.1 Objectif

Vérifier que le fichier **Life_Style_Data.csv** est correctement chargé et obtenir une vue d'ensemble du dataset utilisé pour prédire **Experience_Level**.

Cette étape permet de contrôler :

- les **dimensions** du jeu de données (lignes, colonnes retenues),
- le **typage** des variables (numériques, catégorielles),
- la **présence de valeurs manquantes** ou de doublons,
- la **cohérence générale** du corpus avant l'analyse statistique et la modélisation.

L'objectif final est d'assurer que les features X et la target Y validées sont propres, cohérentes et prêtes pour les étapes d'analyse et de modélisation supervisée.

1.2.2 Interprétation attendue

- Le dataset doit présenter un nombre limité et cohérent de colonnes, correspondant aux **variables retenues pour la modélisation** : données physiques (`Age`, `Weight (kg)`, `Height (m)`), données comportementales (`Workout_Type`, `Workout_Frequency (days/week)`), et

attributs démographiques (Gender).

- Les types doivent être correctement reconnus :
 - **numériques** : Age, Weight (kg), Height (m), Workout_Frequency (days/week)
 - **catégorielles** : Gender, Workout_Type
 - **target numérique** : Experience_Level
- Les colonnes présentant des **valeurs manquantes** devront être identifiées pour planifier leur traitement (suppression, imputation ou encodage).
- La recherche de **doublons** permet d'évaluer la qualité et la diversité des observations du dataset.
- L'inspection des noms de colonnes garantit la cohérence du dictionnaire de données avant d'entamer les analyses statistiques et la modélisation supervisée.

1.2.3 Importation des librairies essentielles

```
[44]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub

from scipy import stats
from scipy.stats import zscore
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import prince

# === Contexte ===
# EDA du dataset Life_Style_Data.csv
# Objectif : analyser les features X et la target Y (Experience_Level)
# utilisées dans le modèle supervisé.

# === Localiser automatiquement le package 'themes' en remontant l'arborescence
↳===
from pathlib import Path
import sys

root = Path.cwd()          # ex: .../DuckDB/eda/life_style_data
themes_root = None
```

```

for p in [root, *root.parents]:    # remonte: life_style_data -> eda -> DuckDB
    ↪-> ...
    if (p / "themes").is_dir():
        themes_root = p
        break

if themes_root is None:
    raise FileNotFoundError(
        f"Impossible de trouver le dossier 'themes' en partant de {root}. "
    )

if str(themes_root) not in sys.path:
    sys.path.insert(0, str(themes_root))

print("Ajouté au PYTHONPATH :", themes_root)
print("Contenu de", themes_root, ":", [x.name for x in (themes_root).iterdir()])

# Thème personnalisé du projet (uniquement pour les notebooks)
from themes.theme_train_me import set_trainme_theme

# Appliquer le thème graphique TrAIIn.me
set_trainme_theme()

```

Ajouté au PYTHONPATH :

c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks

Contenu de c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks :

```

['dataset_fusion', 'eda', 'model_training', 'preprocessing', 'themes',
'__init__.py', '__pycache__']

```

[44]: ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']

1.2.4 Configuration d’affichage pour plus de lisibilité

```

[45]: pd.set_option('display.max_columns', None)
pd.set_option('display.width', 120)

```

1.2.5 Thème “TrAIIn.me”

Il applique un fond dark propre, une grille discrète, des ticks lisibles, et une palette bleus/cians cohérente.

```

[46]: # =====
# Thème TrAIIn.me (Seaborn/Matplotlib)
# =====
# Booléen de contrôle : True = thème TrAIIn.me (dark), False = style clair
    ↪ "ticks"
USE_DARK_THEME = True # modifie ici selon le contexte

```

```
# Activer le thème au début du notebook :
palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
↪use_dark=True)
palette_trainme
```

[46]: ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']

1.2.6 Chargement du dataset

```
[47]: # === Téléchargement du dataset depuis Kaggle ===
dataset_path = kagglehub.dataset_download("jockeroika/life-style-data")

# Recherche du fichier CSV dans le dossier téléchargé
csv_files = [f for f in os.listdir(dataset_path) if f.endswith(".csv")]
print("Fichiers trouvés :", csv_files)

# Construction du chemin complet vers le CSV
file_path = os.path.join(dataset_path, csv_files[0])

# Lecture du fichier CSV brut (avant sélection des features)
df = pd.read_csv(file_path)

# Confirmation
print("Dataset chargé avec succès !")
print(f"Dimensions : {df.shape[0]} lignes et {df.shape[1]} colonnes\n")

print("Aperçu des colonnes originales :")
print(df.columns.tolist())
```

Fichiers trouvés : ['Final_data.csv', 'meal_metadata.csv']

Dataset chargé avec succès !

Dimensions : 20000 lignes et 54 colonnes

Aperçu des colonnes originales :

```
['Age', 'Gender', 'Weight (kg)', 'Height (m)', 'Max_BPM', 'Avg_BPM',
'Resting_BPM', 'Session_Duration (hours)', 'Calories_Burned', 'Workout_Type',
'Fat_Percentage', 'Water_Intake (liters)', 'Workout_Frequency (days/week)',
'Experience_Level', 'BMI', 'Daily meals frequency', 'Physical exercise',
'Carbs', 'Proteins', 'Fats', 'Calories', 'meal_name', 'meal_type', 'diet_type',
'sugar_g', 'sodium_mg', 'cholesterol_mg', 'serving_size_g', 'cooking_method',
'prep_time_min', 'cook_time_min', 'rating', 'Name of Exercise', 'Sets', 'Reps',
'Benefit', 'Burns Calories (per 30 min)', 'Target Muscle Group', 'Equipment
Needed', 'Difficulty Level', 'Body Part', 'Type of Muscle', 'Workout',
'BMI_calc', 'cal_from_macros', 'pct_carbs', 'protein_per_kg', 'pct_HRR',
'pct_maxHR', 'cal_balance', 'lean_mass_kg', 'expected_burn', 'Burns Calories
(per 30 min)_bc', 'Burns_Calories_Bin']
```

1.2.7 Aperçu général des premières lignes

Cette section permet de visualiser immédiatement la structure du dataset, vérifier la présence des colonnes attendues et confirmer que les types semblent cohérents avant d’aller plus loin dans l’analyse.

```
[48]: display(df.head())
```

```
      Age  Gender  Weight (kg)  Height (m)  Max_BPM  Avg_BPM  Resting_BPM  \
↳Session_Duration (hours)  Calories_Burned  \
0  34.91    Male      65.27      1.62    188.58    157.65      69.05      \
↳      1.00      1080.90
1  23.37   Female      56.41      1.55    179.43    131.75      73.18      \
↳      1.37      1809.91
2  33.20   Female      58.98      1.67    175.04    123.95      54.96      \
↳      0.91      802.26
3  38.69   Female      93.78      1.70    191.21    155.10      50.07      \
↳      1.10      1450.79
4  45.09    Male      52.42      1.88    193.58    152.88      70.84      \
↳      1.08      1166.40

      Workout_Type  Fat_Percentage  Water_Intake (liters)  Workout_Frequency (days/
↳week)  Experience_Level  BMI  \
0      Strength      26.800377      1.50      \
↳3.99      2.01  24.87
1           HIIT      27.655021      1.90      \
↳4.00      2.01  23.48
2           Cardio      24.320821      1.88      \
↳2.99      1.02  21.15
3           HIIT      32.813572      2.50      \
↳3.99      1.99  32.45
4      Strength      17.307319      2.91      \
↳4.00      2.00  14.83

      Daily meals frequency  Physical exercise  Carbs  Proteins  Fats  Calories
↳meal_name  meal_type  diet_type  \
0      2.99      0.01  267.68    106.05  71.63    1806.0  \
↳ Other      Lunch      Vegan
1      3.01      0.97  214.32     85.41  56.97    1577.0  \
↳ Other      Lunch  Vegetarian
2      1.99     -0.02  246.04     98.11  65.48    1608.0  \
↳ Other  Breakfast      Paleo
3      3.00      0.04  203.22     80.84  54.56    2657.0  \
↳ Other      Lunch      Paleo
4      3.00      3.00  332.79    133.05  88.43    1470.0  \
↳ Other  Breakfast      Vegan
```

	sugar_g	sodium_mg	cholesterol_mg	serving_size_g	cooking_method	
0	31.77	1729.94	285.05	120.47	Grilled	16.
↳24		110.79	1.31			
1	12.34	693.08	300.61	109.15	Fried	16.
↳47		12.01	1.92			
2	42.81	2142.48	215.42	399.43	Boiled	54.
↳35		6.09	4.70			
3	9.34	123.20	9.70	314.31	Fried	27.
↳73		103.72	4.85			
4	23.78	1935.11	116.89	99.22	Baked	34.
↳16		46.55	3.07			

	Name of Exercise	Sets	Reps	
0	Decline Push-ups	4.99	20.91	Improves shoulder health and posture
↳				342.58
1	Bear Crawls	4.01	16.15	Strengthens lower abs
↳				357.16
2	Dips	5.00	21.90	Builds chest strength
↳				359.63
3	Mountain Climbers	4.01	16.92	Improves coordination and cardiovascular health
↳				351.65
4	Bicep Curls	4.99	15.01	Targets obliques and improves core rotation
↳				329.36

	Target Muscle Group	Equipment Needed	Difficulty Level	Body Part
0	Shoulders, Triceps	Cable Machine	Advanced	Legs
↳	Lats	Dumbbell flyes		
1	Back, Core, Shoulders	Step or Box	Intermediate	Chest
↳	Lats	Lateral raises		
2	Quadriceps, Glutes	Step or Box	Intermediate	Arms
↳	Grip Strength	Standing calf raises		
3	Biceps, Forearms	Parallel Bars or Chair	Advanced	Shoulders
↳	Upper	Incline dumbbell flyes		
4	Chest, Triceps	Wall	Advanced	Abs
↳	Wrist Flexors	Military press		

	BMI_calc	cal_from_macros	pct_carbs	protein_per_kg	pct_HRR	pct_maxHR
0	24.870447	2139.59	0.500432	1.624789	0.741237	0.835985
↳	725.10	47.777394				
1	23.479709	1711.65	0.500850	1.514093	0.551247	0.734270
↳	-232.91	40.809803				

2	21.148123	1965.92	0.500610	1.663445	0.574534	0.708124	␣
↪	805.74	44.635580					
3	32.449827	1627.28	0.499533	0.862017	0.744155	0.811150	␣
↪	1206.21	63.007432					
4	14.831372	2659.23	0.500581	2.538153	0.668405	0.789751	␣
↪	303.60	43.347504					

	expected_burn	Burns Calories (per 30 min)_bc	Burns_Calories_Bin
0	685.1600	7.260425e+19	Medium
1	978.6184	1.020506e+20	High
2	654.5266	1.079607e+20	High
3	773.6300	8.987921e+19	High
4	711.4176	5.264685e+19	Low

1.2.8 Informations générales sur le typage et la complétude

Cette section vérifie la nature des colonnes (numériques ou catégorielles), détecte d'éventuelles valeurs manquantes et assure que le dataset est suffisamment propre pour poursuivre l'analyse.

```
[49]: df_info = df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 54 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   20000 non-null  float64
1   Gender                               20000 non-null  object
2   Weight (kg)                          20000 non-null  float64
3   Height (m)                           20000 non-null  float64
4   Max_BPM                              20000 non-null  float64
5   Avg_BPM                              20000 non-null  float64
6   Resting_BPM                          20000 non-null  float64
7   Session_Duration (hours)             20000 non-null  float64
8   Calories_Burned                       20000 non-null  float64
9   Workout_Type                          20000 non-null  object
10  Fat_Percentage                        20000 non-null  float64
11  Water_Intake (liters)                 20000 non-null  float64
12  Workout_Frequency (days/week)        20000 non-null  float64
13  Experience_Level                      20000 non-null  float64
14  BMI                                   20000 non-null  float64
15  Daily meals frequency                 20000 non-null  float64
16  Physical exercise                     20000 non-null  float64
17  Carbs                                 20000 non-null  float64
18  Proteins                              20000 non-null  float64
19  Fats                                  20000 non-null  float64
20  Calories                              20000 non-null  float64
21  meal_name                             20000 non-null  object
```

```

22 meal_type                20000 non-null object
23 diet_type                20000 non-null object
24 sugar_g                  20000 non-null float64
25 sodium_mg                20000 non-null float64
26 cholesterol_mg           20000 non-null float64
27 serving_size_g           20000 non-null float64
28 cooking_method            20000 non-null object
29 prep_time_min             20000 non-null float64
30 cook_time_min             20000 non-null float64
31 rating                    20000 non-null float64
32 Name of Exercise          20000 non-null object
33 Sets                      20000 non-null float64
34 Reps                      20000 non-null float64
35 Benefit                   20000 non-null object
36 Burns Calories (per 30 min) 20000 non-null float64
37 Target Muscle Group       20000 non-null object
38 Equipment Needed          20000 non-null object
39 Difficulty Level          20000 non-null object
40 Body Part                 20000 non-null object
41 Type of Muscle            20000 non-null object
42 Workout                   20000 non-null object
43 BMI_calc                  20000 non-null float64
44 cal_from_macros           20000 non-null float64
45 pct_carbs                 20000 non-null float64
46 protein_per_kg            20000 non-null float64
47 pct_HRR                   20000 non-null float64
48 pct_maxHR                 20000 non-null float64
49 cal_balance               20000 non-null float64
50 lean_mass_kg              20000 non-null float64
51 expected_burn             20000 non-null float64
52 Burns Calories (per 30 min)_bc 20000 non-null float64
53 Burns_Calories_Bin        20000 non-null object
dtypes: float64(39), object(15)
memory usage: 8.2+ MB

```

1.2.9 Statistiques descriptives des variables numériques

Cette section synthétise les mesures essentielles (moyenne, dispersion, asymétrie, valeurs manquantes, etc.) afin de comprendre la distribution et la variabilité des variables numériques retenues.

```
[50]: display(df.describe().T)
```

	count	mean	std	
min	25%	50%	\	
Age	20000.0	3.885145e+01	1.211458e+01	1.
800000e+01	2.817000e+01	3.986500e+01		
Weight (kg)	20000.0	7.389883e+01	2.117301e+01	3.
918000e+01	5.816000e+01	7.000000e+01		

Height (m)	20000.0	1.723094e+00	1.270328e-01	1.
↪490000e+00	1.620000e+00	1.710000e+00		
Max_BPM	20000.0	1.798897e+02	1.151081e+01	1.
↪593100e+02	1.700575e+02	1.801400e+02		
Avg_BPM	20000.0	1.437043e+02	1.426769e+01	1.
↪190700e+02	1.312200e+02	1.429900e+02		
Resting_BPM	20000.0	6.219581e+01	7.289351e+00	4.
↪949000e+01	5.596000e+01	6.220000e+01		
Session_Duration (hours)	20000.0	1.259446e+00	3.413362e-01	4.
↪900000e-01	1.050000e+00	1.270000e+00		
Calories_Burned	20000.0	1.280110e+03	5.022290e+02	3.
↪231100e+02	9.108000e+02	1.231450e+03		
Fat_Percentage	20000.0	2.610131e+01	4.996160e+00	1.
↪133313e+01	2.238781e+01	2.582250e+01		
Water_Intake (liters)	20000.0	2.627486e+00	6.047240e-01	1.
↪460000e+00	2.170000e+00	2.610000e+00		
Workout_Frequency (days/week)	20000.0	3.318629e+00	9.111979e-01	1.
↪940000e+00	2.980000e+00	3.010000e+00		
Experience_Level	20000.0	1.809176e+00	7.362036e-01	1.
↪000000e+00	1.010000e+00	1.990000e+00		
BMI	20000.0	2.492168e+01	6.701126e+00	1.
↪204000e+01	2.009750e+01	2.412000e+01		
Daily meals frequency	20000.0	2.864804e+00	6.366486e-01	1.
↪950000e+00	2.020000e+00	3.000000e+00		
Physical exercise	20000.0	4.525030e-01	9.866276e-01	-7.
↪000000e-02	-1.000000e-02	1.000000e-02		
Carbs	20000.0	2.497786e+02	5.510862e+01	1.
↪377200e+02	2.075475e+02	2.429000e+02		
Proteins	20000.0	9.991829e+01	2.204367e+01	5.
↪517000e+01	8.304000e+01	9.714500e+01		
Fats	20000.0	6.661217e+01	1.469928e+01	3.
↪659000e+01	5.534000e+01	6.477000e+01		
Calories	20000.0	2.024418e+03	5.418915e+02	7.
↪810000e+02	1.634000e+03	1.919000e+03		
sugar_g	20000.0	2.453104e+01	1.444610e+01	-6.
↪700000e-01	1.199000e+01	2.406000e+01		
sodium_mg	20000.0	1.258140e+03	7.166190e+02	1.
↪922000e+01	6.451275e+02	1.267650e+03		
cholesterol_mg	20000.0	1.484656e+02	8.738282e+01	-3.
↪890000e+00	7.197000e+01	1.497950e+02		
serving_size_g	20000.0	3.027195e+02	1.157119e+02	9.
↪595000e+01	2.072175e+02	3.000150e+02		
prep_time_min	20000.0	3.351745e+01	1.652687e+01	3.
↪950000e+00	1.858750e+01	3.433000e+01		
cook_time_min	20000.0	6.150216e+01	3.357975e+01	3.
↪350000e+00	3.247750e+01	6.092000e+01		

rating	20000.0	2.989303e+00	1.157692e+00	9.
↪300000e-01	1.940000e+00	3.000000e+00		
Sets	20000.0	4.425059e+00	5.795959e-01	2.
↪960000e+00	4.000000e+00	4.020000e+00		
Reps	20000.0	1.942732e+01	3.679707e+00	4.
↪850000e+00	1.612000e+01	1.990000e+01		
Burns Calories (per 30 min)	20000.0	3.440465e+02	3.213878e+01	1.
↪287500e+02	3.376000e+02	3.486050e+02		
BMI_calc	20000.0	2.492165e+01	6.701144e+00	1.
↪203791e+01	2.009497e+01	2.411910e+01		
cal_from_macros	20000.0	1.998297e+03	4.408484e+02	1.
↪105570e+03	1.661023e+03	1.943130e+03		
pct_carbs	20000.0	4.999830e-01	1.455490e-03	4.
↪924342e-01	4.990538e-01	4.999810e-01		
protein_per_kg	20000.0	1.460142e+00	5.189455e-01	5.
↪167064e-01	1.076294e+00	1.382260e+00		
pct_HRR	20000.0	6.990053e-01	1.448804e-01	3.
↪713439e-01	5.836562e-01	6.862837e-01		
pct_maxHR	20000.0	8.023050e-01	9.661268e-02	5.
↪997889e-01	7.276759e-01	7.948340e-01		
cal_balance	20000.0	7.443087e+02	7.209466e+02	-1.
↪266220e+03	2.614325e+02	6.911900e+02		
lean_mass_kg	20000.0	5.378638e+01	1.249874e+01	3.
↪094626e+01	4.458704e+01	5.120491e+01		
expected_burn	20000.0	8.663523e+02	2.503171e+02	2.
↪198528e+02	7.140983e+02	8.687214e+02		
Burns Calories (per 30 min)_bc	20000.0	8.631802e+19	3.197579e+19	2.
↪491905e+16	6.441978e+19	8.371578e+19		

	75%	max
Age	4.963000e+01	5.967000e+01
Weight (kg)	8.610000e+01	1.307700e+02
Height (m)	1.800000e+00	2.010000e+00
Max_BPM	1.894250e+02	1.996400e+02
Avg_BPM	1.560600e+02	1.698400e+02
Resting_BPM	6.809000e+01	7.450000e+01
Session_Duration (hours)	1.460000e+00	2.020000e+00
Calories_Burned	1.553112e+03	2.890820e+03
Fat_Percentage	2.967603e+01	3.500000e+01
Water_Intake (liters)	3.120000e+00	3.730000e+00
Workout_Frequency (days/week)	4.000000e+00	5.060000e+00
Experience_Level	2.020000e+00	3.050000e+00
BMI	2.856000e+01	5.023000e+01
Daily meals frequency	3.010000e+00	4.040000e+00
Physical exercise	4.000000e-02	4.050000e+00
Carbs	2.839750e+02	4.624900e+02

Proteins	1.136400e+02	1.853400e+02
Fats	7.575250e+01	1.234200e+02
Calories	2.360000e+03	3.641000e+03
sugar_g	3.749000e+01	5.051000e+01
sodium_mg	1.850893e+03	2.527270e+03
cholesterol_mg	2.218400e+02	3.039900e+02
serving_size_g	4.018600e+02	5.075200e+02
prep_time_min	4.794000e+01	6.129000e+01
cook_time_min	8.937500e+01	1.214600e+02
rating	4.000000e+00	5.060000e+00
Sets	5.000000e+00	5.050000e+00
Reps	2.288000e+01	3.012000e+01
Burns Calories (per 30 min)	3.604725e+02	3.817100e+02
BMI_calc	2.856262e+01	5.022954e+01
cal_from_macros	2.271950e+03	3.699540e+03
pct_carbs	5.009103e-01	5.078886e-01
protein_per_kg	1.750495e+00	3.916881e+00
pct_HRR	7.981957e-01	1.073939e+00
pct_maxHR	8.692108e-01	1.047032e+00
cal_balance	1.176290e+03	3.075580e+03
lean_mass_kg	6.193902e+01	9.011737e+01
expected_burn	1.012533e+03	1.477109e+03
Burns Calories (per 30 min)_bc	1.100442e+20	1.756614e+20

1.2.10 Analyse des valeurs manquantes et doublons

Cette section permet d'identifier les colonnes pouvant nécessiter un nettoyage ou une imputation, ainsi que la présence de doublons susceptibles d'altérer la qualité du dataset ou de biaiser la modélisation.

```
[51]: missing_values = df.isnull().sum().sort_values(ascending=False)
      duplicates = df.duplicated().sum()

      print("\n--- Valeurs manquantes par colonne (top 10) ---")
      display(missing_values.head(10))

      print(f"\nNombre total de doublons détectés : {duplicates}")
```

```
--- Valeurs manquantes par colonne (top 10) ---
```

Age	0
Body Part	0
prep_time_min	0
cook_time_min	0
rating	0
Name of Exercise	0
Sets	0
Reps	0

```
Benefit                                0
Burns Calories (per 30 min)           0
dtype: int64
```

Nombre total de doublons détectés : 0

1.2.11 Liste complète des colonnes disponibles

Cette section offre une vue d'ensemble de toutes les variables présentes dans le dataset afin de vérifier leur cohérence, repérer d'éventuelles colonnes inutiles et confirmer la présence des features attendues pour la modélisation.

```
[52]: print("\n--- Liste des colonnes du dataset ---")
      print(df.columns.tolist())
```

```
--- Liste des colonnes du dataset ---
['Age', 'Gender', 'Weight (kg)', 'Height (m)', 'Max_BPM', 'Avg_BPM',
'Resting_BPM', 'Session_Duration (hours)', 'Calories_Burned', 'Workout_Type',
'Fat_Percentage', 'Water_Intake (liters)', 'Workout_Frequency (days/week)',
'Experience_Level', 'BMI', 'Daily meals frequency', 'Physical exercise',
'Carbs', 'Proteins', 'Fats', 'Calories', 'meal_name', 'meal_type', 'diet_type',
'sugar_g', 'sodium_mg', 'cholesterol_mg', 'serving_size_g', 'cooking_method',
'prep_time_min', 'cook_time_min', 'rating', 'Name of Exercise', 'Sets', 'Reps',
'Benefit', 'Burns Calories (per 30 min)', 'Target Muscle Group', 'Equipment
Needed', 'Difficulty Level', 'Body Part', 'Type of Muscle', 'Workout',
'BMI_calc', 'cal_from_macros', 'pct_carbs', 'protein_per_kg', 'pct_HRR',
'pct_maxHR', 'cal_balance', 'lean_mass_kg', 'expected_burn', 'Burns Calories
(per 30 min)_bc', 'Burns_Calories_Bin']
```

1.3 Statistiques descriptives et distribution des variables

1.3.1 Objectif

Explorer la distribution des variables numériques afin de comprendre la variabilité du dataset, identifier d'éventuelles anomalies et détecter les premiers signaux statistiques utiles.

1.3.2 Interprétation attendue

- Confirmer la cohérence statistique des variables physiologiques (âge, poids, taille, IMC).
- Vérifier la normalité ou l'asymétrie de certaines distributions (Calories_Burned, BMI...).
- Identifier visuellement les outliers potentiels pour un traitement futur.

1.3.3 Activation du thème TrAIIn.me

```
[53]: if USE_DARK_THEME:
    palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
    ↪ use_dark=True)
else:
    sns.set_style("ticks") # style clair sobre, axes renforcés
    sns.set_context("talk", font_scale=1.05)
    palette_trainme = sns.color_palette("deep") # palette par défaut sobre
    print("Style activé : 'ticks' → sobre, axes renforcés (présentation pro)")

# Exemple de vérification :
print("Palette active :", palette_trainme.as_hex() if hasattr(palette_trainme,
    ↪ "as_hex") else palette_trainme)
```

Palette active : ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']

1.3.4 Statistiques descriptives globales

Cette section présente un résumé statistique de l'ensemble des colonnes numériques du dataset afin d'obtenir une vision rapide de leurs tendances générales, amplitudes et dispersions avant d'approfondir l'analyse variable par variable.

```
[54]: stats = df.describe().T
display(stats)
# Vérification des bornes extrêmes sur les principales métriques
print(f"Âge moyen : {df['Age'].mean():.1f} ans")
print(f"Poids moyen : {df['Weight (kg)'].mean():.1f} kg")
print(f"Calories moyennes brûlées : {df['Calories_Burned'].mean():.0f} kcal")
print(f"IMC moyen : {df['BMI'].mean():.1f}")
```

	count	mean	std	
↪ min	25%	50% \		
Age	20000.0	3.885145e+01	1.211458e+01	1.
↪ 800000e+01	2.817000e+01	3.986500e+01		
Weight (kg)	20000.0	7.389883e+01	2.117301e+01	3.
↪ 918000e+01	5.816000e+01	7.000000e+01		
Height (m)	20000.0	1.723094e+00	1.270328e-01	1.
↪ 490000e+00	1.620000e+00	1.710000e+00		
Max_BPM	20000.0	1.798897e+02	1.151081e+01	1.
↪ 593100e+02	1.700575e+02	1.801400e+02		
Avg_BPM	20000.0	1.437043e+02	1.426769e+01	1.
↪ 190700e+02	1.312200e+02	1.429900e+02		
Resting_BPM	20000.0	6.219581e+01	7.289351e+00	4.
↪ 949000e+01	5.596000e+01	6.220000e+01		
Session_Duration (hours)	20000.0	1.259446e+00	3.413362e-01	4.
↪ 900000e-01	1.050000e+00	1.270000e+00		

Calories_Burned	20000.0	1.280110e+03	5.022290e+02	3.
↳231100e+02	9.108000e+02	1.231450e+03		
Fat_Percentage	20000.0	2.610131e+01	4.996160e+00	1.
↳133313e+01	2.238781e+01	2.582250e+01		
Water_Intake (liters)	20000.0	2.627486e+00	6.047240e-01	1.
↳460000e+00	2.170000e+00	2.610000e+00		
Workout_Frequency (days/week)	20000.0	3.318629e+00	9.111979e-01	1.
↳940000e+00	2.980000e+00	3.010000e+00		
Experience_Level	20000.0	1.809176e+00	7.362036e-01	1.
↳000000e+00	1.010000e+00	1.990000e+00		
BMI	20000.0	2.492168e+01	6.701126e+00	1.
↳204000e+01	2.009750e+01	2.412000e+01		
Daily meals frequency	20000.0	2.864804e+00	6.366486e-01	1.
↳950000e+00	2.020000e+00	3.000000e+00		
Physical exercise	20000.0	4.525030e-01	9.866276e-01	-7.
↳000000e-02	-1.000000e-02	1.000000e-02		
Carbs	20000.0	2.497786e+02	5.510862e+01	1.
↳377200e+02	2.075475e+02	2.429000e+02		
Proteins	20000.0	9.991829e+01	2.204367e+01	5.
↳517000e+01	8.304000e+01	9.714500e+01		
Fats	20000.0	6.661217e+01	1.469928e+01	3.
↳659000e+01	5.534000e+01	6.477000e+01		
Calories	20000.0	2.024418e+03	5.418915e+02	7.
↳810000e+02	1.634000e+03	1.919000e+03		
sugar_g	20000.0	2.453104e+01	1.444610e+01	-6.
↳700000e-01	1.199000e+01	2.406000e+01		
sodium_mg	20000.0	1.258140e+03	7.166190e+02	1.
↳922000e+01	6.451275e+02	1.267650e+03		
cholesterol_mg	20000.0	1.484656e+02	8.738282e+01	-3.
↳890000e+00	7.197000e+01	1.497950e+02		
serving_size_g	20000.0	3.027195e+02	1.157119e+02	9.
↳595000e+01	2.072175e+02	3.000150e+02		
prep_time_min	20000.0	3.351745e+01	1.652687e+01	3.
↳950000e+00	1.858750e+01	3.433000e+01		
cook_time_min	20000.0	6.150216e+01	3.357975e+01	3.
↳350000e+00	3.247750e+01	6.092000e+01		
rating	20000.0	2.989303e+00	1.157692e+00	9.
↳300000e-01	1.940000e+00	3.000000e+00		
Sets	20000.0	4.425059e+00	5.795959e-01	2.
↳960000e+00	4.000000e+00	4.020000e+00		
Reps	20000.0	1.942732e+01	3.679707e+00	4.
↳850000e+00	1.612000e+01	1.990000e+01		
Burns Calories (per 30 min)	20000.0	3.440465e+02	3.213878e+01	1.
↳287500e+02	3.376000e+02	3.486050e+02		
BMI_calc	20000.0	2.492165e+01	6.701144e+00	1.
↳203791e+01	2.009497e+01	2.411910e+01		

cal_from_macros	20000.0	1.998297e+03	4.408484e+02	1.
↪105570e+03	1.661023e+03	1.943130e+03		
pct_carbs	20000.0	4.999830e-01	1.455490e-03	4.
↪924342e-01	4.990538e-01	4.999810e-01		
protein_per_kg	20000.0	1.460142e+00	5.189455e-01	5.
↪167064e-01	1.076294e+00	1.382260e+00		
pct_HRR	20000.0	6.990053e-01	1.448804e-01	3.
↪713439e-01	5.836562e-01	6.862837e-01		
pct_maxHR	20000.0	8.023050e-01	9.661268e-02	5.
↪997889e-01	7.276759e-01	7.948340e-01		
cal_balance	20000.0	7.443087e+02	7.209466e+02	-1.
↪266220e+03	2.614325e+02	6.911900e+02		
lean_mass_kg	20000.0	5.378638e+01	1.249874e+01	3.
↪094626e+01	4.458704e+01	5.120491e+01		
expected_burn	20000.0	8.663523e+02	2.503171e+02	2.
↪198528e+02	7.140983e+02	8.687214e+02		
Burns Calories (per 30 min)_bc	20000.0	8.631802e+19	3.197579e+19	2.
↪491905e+16	6.441978e+19	8.371578e+19		

	75%	max
Age	4.963000e+01	5.967000e+01
Weight (kg)	8.610000e+01	1.307700e+02
Height (m)	1.800000e+00	2.010000e+00
Max_BPM	1.894250e+02	1.996400e+02
Avg_BPM	1.560600e+02	1.698400e+02
Resting_BPM	6.809000e+01	7.450000e+01
Session_Duration (hours)	1.460000e+00	2.020000e+00
Calories_Burned	1.553112e+03	2.890820e+03
Fat_Percentage	2.967603e+01	3.500000e+01
Water_Intake (liters)	3.120000e+00	3.730000e+00
Workout_Frequency (days/week)	4.000000e+00	5.060000e+00
Experience_Level	2.020000e+00	3.050000e+00
BMI	2.856000e+01	5.023000e+01
Daily meals frequency	3.010000e+00	4.040000e+00
Physical exercise	4.000000e-02	4.050000e+00
Carbs	2.839750e+02	4.624900e+02
Proteins	1.136400e+02	1.853400e+02
Fats	7.575250e+01	1.234200e+02
Calories	2.360000e+03	3.641000e+03
sugar_g	3.749000e+01	5.051000e+01
sodium_mg	1.850893e+03	2.527270e+03
cholesterol_mg	2.218400e+02	3.039900e+02
serving_size_g	4.018600e+02	5.075200e+02
prep_time_min	4.794000e+01	6.129000e+01
cook_time_min	8.937500e+01	1.214600e+02
rating	4.000000e+00	5.060000e+00
Sets	5.000000e+00	5.050000e+00

Reps	2.288000e+01	3.012000e+01
Burns Calories (per 30 min)	3.604725e+02	3.817100e+02
BMI_calc	2.856262e+01	5.022954e+01
cal_from_macros	2.271950e+03	3.699540e+03
pct_carbs	5.009103e-01	5.078886e-01
protein_per_kg	1.750495e+00	3.916881e+00
pct_HRR	7.981957e-01	1.073939e+00
pct_maxHR	8.692108e-01	1.047032e+00
cal_balance	1.176290e+03	3.075580e+03
lean_mass_kg	6.193902e+01	9.011737e+01
expected_burn	1.012533e+03	1.477109e+03
Burns Calories (per 30 min)_bc	1.100442e+20	1.756614e+20

Âge moyen : 38.9 ans
 Poids moyen : 73.9 kg
 Calories moyennes brûlées : 1280 kcal
 IMC moyen : 24.9

1.3.5 Distribution des variables clés

Cette section explore la répartition des principales variables numériques afin d'identifier leur comportement global, repérer d'éventuelles asymétries et visualiser la variabilité des données utilisées pour la modélisation.

```
[55]: # Variables numériques définies par le schéma JSON
num_vars = [
    "Age",
    "Weight (kg)",
    "Height (m)",
    "Workout_Frequency (days/week)",
    "Experience_Level", # target
]

# Vérification : ne garder que celles présentes dans le df
num_vars = [col for col in num_vars if col in df.columns]

print("Variables numériques utilisées :")
print(num_vars)

n_cols = 3
n_rows = int(np.ceil(len(num_vars) / n_cols))

plt.figure(figsize=(5 * n_cols, 4 * n_rows))
for i, col in enumerate(num_vars, 1):
    plt.subplot(n_rows, n_cols, i)
    sns.histplot(
        df[col],
        bins=30,
```



```

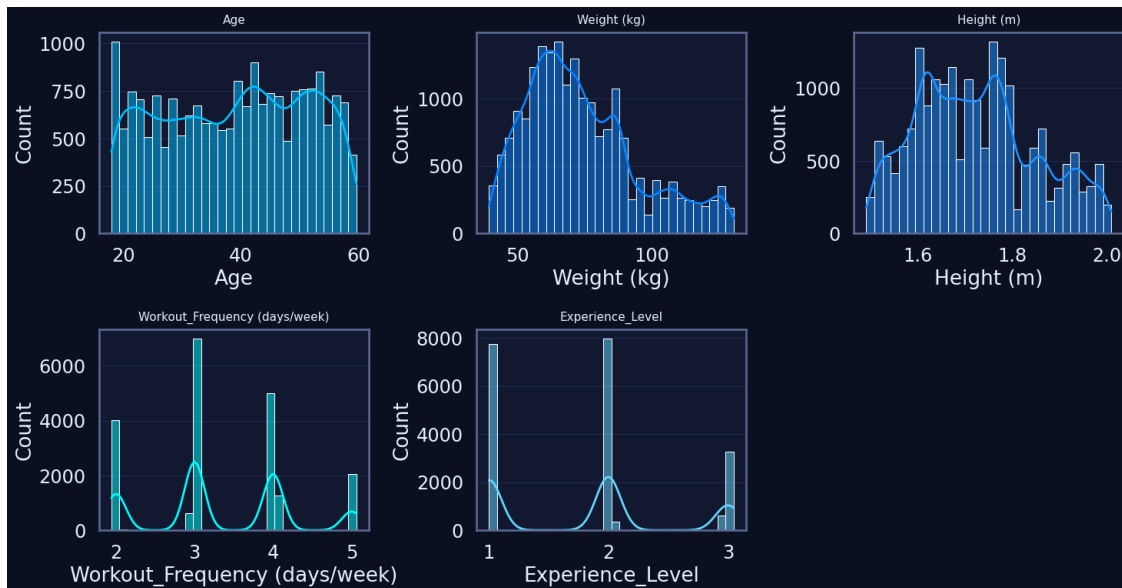
        kde=True,
        color=palette_trainme[i % len(palette_trainme)]
    )
    plt.title(col, fontsize=11, pad=8)

plt.tight_layout()
plt.show()

```

Variables numériques utilisées :

```
['Age', 'Weight (kg)', 'Height (m)', 'Workout_Frequency (days/week)',
'Experience_Level']
```



Interprétations : Age

La distribution est étendue entre 18 et 60 ans, sans concentration excessive sur une tranche précise. La forme légèrement ondulée reflète une population variée plutôt qu'un groupe homogène. Cela indique qu'aucun biais majeur lié à l'âge n'est à prévoir.

Weight (kg)

Le poids suit une distribution asymétrique centrée autour de 70–80 kg, avec une décroissance progressive au-delà de 100 kg.

Quelques valeurs élevées existent mais restent physiologiquement plausibles.

La variabilité est suffisante pour que cette variable contribue de manière utile au modèle.

Height (m)

La taille est principalement comprise entre 1.55 m et 1.90 m, avec un pic autour de 1.72–1.78 m.

Les extrêmes sont rares et cohérents avec une population adulte diversifiée.

Workout_Frequency (days/week)

La distribution est discrète, avec des pics prononcés sur les valeurs 2, 3, 4 et 5 jours/semaine. Cela suggère des comportements d'entraînement structurés autour de catégories ou de routines typiques.

Experience_Level

La variable cible est concentrée autour de trois niveaux principaux (1, 2 et 3), avec une dominance du niveau intermédiaire.

Ce profil est typique d'une population sportive majoritairement régulière mais non experte.

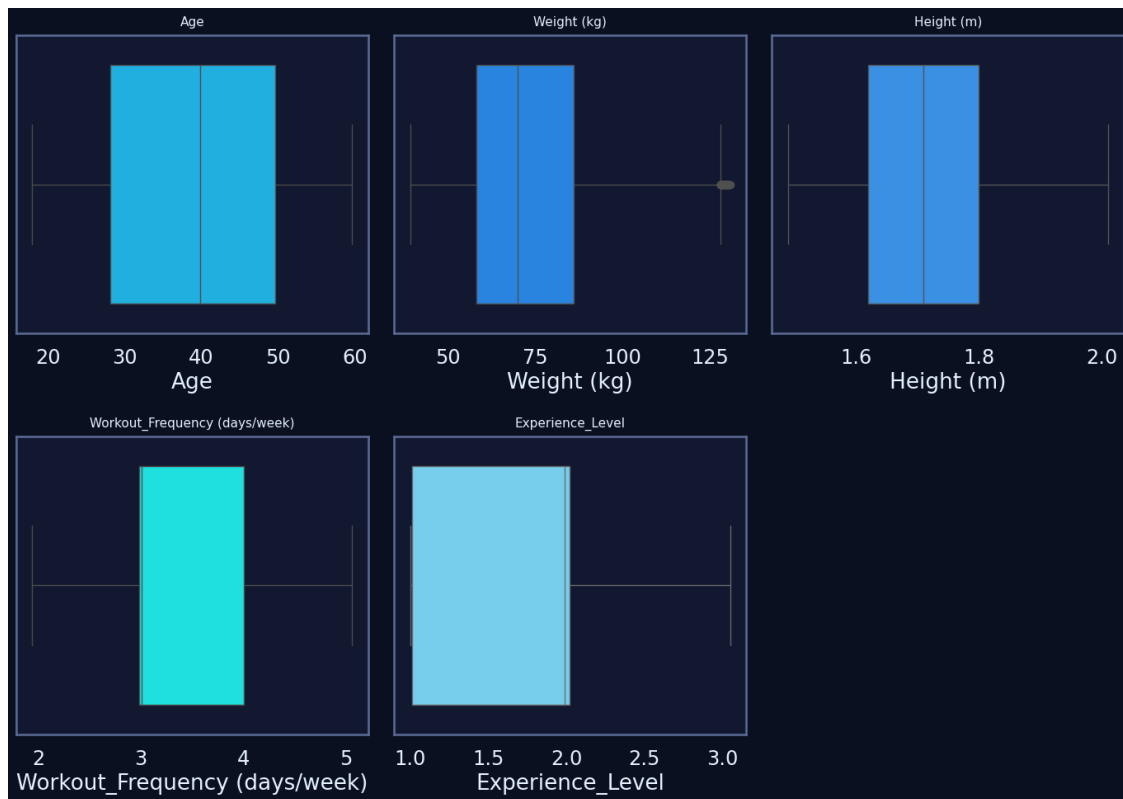
1.3.6 Boxplots pour repérer les outliers

Cette section utilise des boxplots pour visualiser rapidement la présence de valeurs extrêmes, permettant de repérer les observations atypiques susceptibles d'influencer la modélisation.

```
[56]: plt.figure(figsize=(14, 10))
n_cols = 3
n_rows = int(np.ceil(len(num_vars) / n_cols))

for i, col in enumerate(num_vars, 1):
    plt.subplot(n_rows, n_cols, i)
    sns.boxplot(
        x=df[col],
        color=palette_trainme[i % len(palette_trainme)]
    )
    plt.title(col, fontsize=11, pad=8)

plt.tight_layout()
plt.show()
```



Interprétations : Age

La distribution est compacte, centrée autour de valeurs intermédiaires, avec quelques valeurs légèrement inférieures ou supérieures mais pas de véritable outlier.

Le profil est stable et ne présente pas d'écarts extrêmes.

Weight (kg)

La majorité des valeurs est regroupée autour de 70–90 kg.

Un ensemble de valeurs supérieures (>110 kg) apparaît comme outliers, mais elles restent physiologiquement plausibles et reflètent une diversité naturelle de morphologies.

Height (m)

La taille est bien concentrée dans un intervalle serré, sans point aberrant marqué.

Les whiskers s'étendent de manière régulière, montrant des variations normales d'une population adulte.

Workout_Frequency (days/week)

La variable ne présente aucun outlier visuel : la fréquence d'entraînement est répartie de façon nette entre les valeurs 2, 3, 4 et 5 jours/semaine.

La distribution discrète explique l'absence de points isolés.

Experience_Level

Les niveaux d'expérience sont correctement répartis, sans valeur anormale ni point éloigné. La structure en trois niveaux (1, 2, 3) génère des boxplots compacts, cohérents avec une variable catégorisée ordinalement.

1.3.7 Statistiques descriptives enrichies

Cette section complète l'analyse en ajoutant des métriques avancées telles que le pourcentage de valeurs manquantes, la proportion de zéros, l'asymétrie et la kurtosis, offrant une compréhension plus fine de la dynamique des variables numériques.

```
[57]: desc = df[num_vars].describe().T
desc["missing_%"] = df[num_vars].isna().mean() * 100
desc["zeros_%"] = (df[num_vars] == 0).mean() * 100
desc["skew"] = df[num_vars].skew()
desc["kurt"] = df[num_vars].kurt()

display(desc.sort_values("std", ascending=False))
```

	count	mean	std	min	25%	75%	max	missing_%	zeros_%	skew	kurt
Weight (kg)	20000.0	73.898832	21.173010	39.18	58.16	70.00	130.77	0.0	0.0	0.769024	-0.031195
Age	20000.0	38.851454	12.114580	18.00	28.17	39.00	59.67	0.0	0.0	-0.091603	-1.204356
Workout_Frequency (days/week)	20000.0	3.318629	0.911198	1.94	2.98	3.00	5.06	0.0	0.0	0.153587	-0.800936
Experience_Level	20000.0	1.809176	0.736204	1.00	1.01	1.00	3.05	0.0	0.0	0.327854	-1.112786
Height (m)	20000.0	1.723094	0.127033	1.49	1.62	1.70	2.01	0.0	0.0	0.334556	-0.702015

1.3.8 Détection rapide d'outliers (IQR)

Cette section utilise la méthode de l'IQR (Interquartile Range) pour quantifier automatiquement la proportion de valeurs extrêmes par variable, facilitant l'identification des colonnes potentiellement sensibles aux outliers.

```
[58]: def iqr_outlier_ratio(s: pd.Series, k: float = 1.5) -> float:
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    iqr = q3 - q1
    if iqr == 0:
```

```

        return 0.0
    low, high = q1 - k * iqr, q3 + k * iqr
    return ((s < low) | (s > high)).mean() * 100

outlier_report = pd.Series({c: iqr_outlier_ratio(df[c]) for c in num_vars},
    name="outliers_%")
display(outlier_report.sort_values(ascending=False))

```

```

Weight (kg)          0.76
Age                  0.00
Height (m)           0.00
Workout_Frequency (days/week) 0.00
Experience_Level      0.00
Name: outliers_%, dtype: float64

```

1.3.9 Aperçu corrélations (sous-ensemble lisible)

Cette section visualise les corrélations entre les variables essentielles du dataset afin d'identifier rapidement les relations fortes ou faibles qui pourraient influencer la modélisation.

```

[59]: # Sous-ensemble pour la matrice de corrélation (d'après le schéma JSON)
subset_for_corr = [
    "Age",
    "Weight (kg)",
    "Height (m)",
    "Workout_Frequency (days/week)",
    "Experience_Level", # target
]

# Vérification : ne garder que les colonnes présentes dans le DataFrame
subset_for_corr = [c for c in subset_for_corr if c in df.columns]

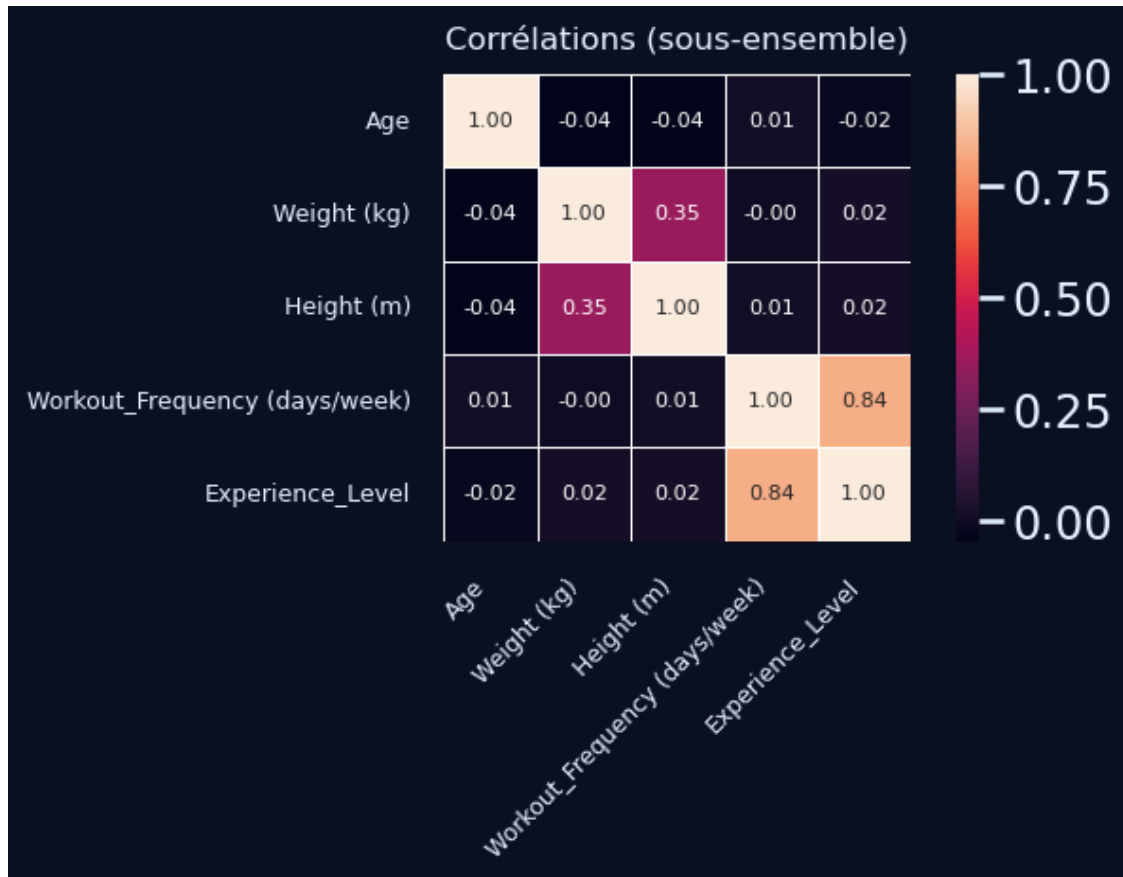
corr = df[subset_for_corr].corr(numeric_only=True)

plt.figure(figsize=(7, 5))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    square=True,
    cbar=True,
    annot_kws={"size": 8},
    linewidths=0.5,
)

plt.title("Corrélations (sous-ensemble)", fontsize=12, pad=10)
plt.xticks(fontsize=9, rotation=45, ha="right")
plt.yticks(fontsize=9)

```

```
plt.tight_layout()
plt.show()
```



Interprétations : Age

Les corrélations liées à l'âge sont proches de zéro, ce qui indique que cette variable n'entretient pas de relation linéaire forte avec les autres paramètres du sous-ensemble.

L'âge ne semble ni influencer, ni être influencé par les habitudes d'entraînement ou l'expérience déclarée.

Weight (kg)

Une corrélation positive modérée apparaît avec la taille (**Height (m)**), logique sur le plan physiologique : les individus plus grands tendent à être plus lourds.

Les autres relations sont négligeables, montrant un poids indépendant de la fréquence d'entraînement ou du niveau d'expérience.

Height (m)

La relation la plus notable reste celle avec le poids (0.35).

Aucune autre corrélation significative n'émerge, confirmant que la taille ne conditionne pas les comportements d'activité physique dans ce dataset.

Workout_Frequency (days/week)

La corrélation la plus marquante est celle avec `Experience_Level` (0.84), très forte et parfaitement cohérente :

plus un individu s'entraîne souvent, plus son niveau d'expérience est élevé.

Experience_Level

La variable cible se distingue clairement par sa relation étroite avec la fréquence d'entraînement. Les autres corrélations sont quasi nulles, suggérant que **l'expérience n'est pas liée aux caractéristiques physiques**, mais principalement aux habitudes sportives.

1.4 Visualisation univariée

1.4.1 Objectif

Analyser séparément la distribution des principales variables numériques afin de : - repérer d'éventuelles asymétries ou valeurs atypiques, - comprendre les profils dominants de la population étudiée, - vérifier la cohérence des mesures nécessaires à la modélisation (âge, poids, taille, fréquence d'entraînement, niveau d'expérience).

1.4.2 Interprétation attendue

- **Age** : distribution généralement centrée, typique d'une population adulte.
- **Weight (kg)** et **Height (m)** : répartition large mais cohérente, reflétant la diversité morphologique des utilisateurs.
- **Workout_Frequency (days/week)** : concentrée autour de valeurs moyennes (population pratiquant quelques séances par semaine).
- **Experience_Level** (*target*) : progression logique d'un niveau novice à avancé, avec une concentration sur les niveaux intermédiaires.

1.4.3 Visualisation des distributions univariées

Cette section examine séparément chaque variable numérique afin d'observer sa forme, sa dispersion et d'éventuelles asymétries, offrant une première compréhension individuelle des comportements présents dans le dataset.

```
[60]: # Variables univariées à visualiser (selon schéma JSON)
uni_vars = [
    "Age",
    "Weight (kg)",
    "Height (m)",
    "Workout_Frequency (days/week)",
    "Experience_Level", # target
]

# Filtrer selon les colonnes réellement présentes
uni_vars = [c for c in uni_vars if c in df.columns]

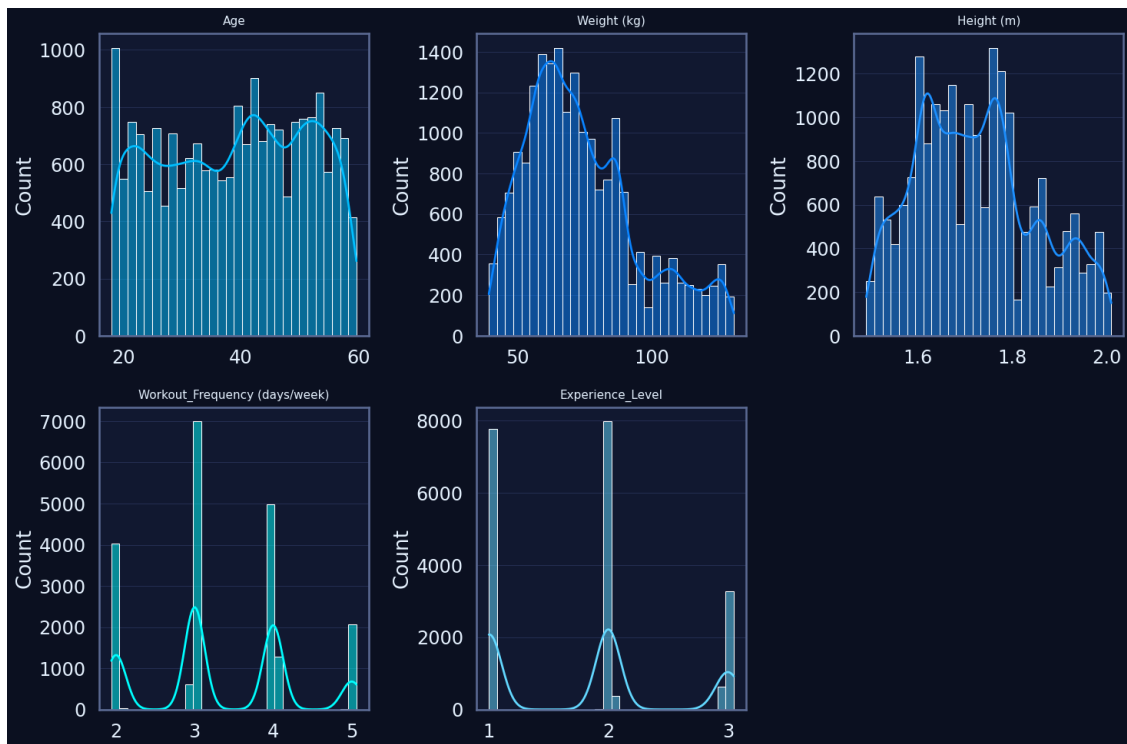
plt.figure(figsize=(15, 10))
n_cols = 3
```

```

n_rows = int(np.ceil(len(uni_vars) / n_cols))

for i, col in enumerate(uni_vars, 1):
    ax = plt.subplot(n_rows, n_cols, i)
    sns.histplot(
        df[col],
        kde=True,
        color=palette_trainme[i % len(palette_trainme)],
        bins=30
    )
    ax.set_title(col, fontsize=11, pad=8)
    ax.set_xlabel("")
plt.tight_layout()
plt.show()

```



Interprétations : Age

La distribution couvre uniformément l'intervalle 18–60 ans, sans pics marqués ni zones creuses. La densité presque plate illustre une forte diversité d'âges, ce qui garantit une bonne représentativité pour le modèle.

Weight (kg)

Le poids montre une distribution asymétrique avec un pic net autour de 70–80 kg, suivi d'une décroissance progressive.

Quelques valeurs au-delà de 110 kg enrichissent la variabilité sans constituer d'anomalies.

Height (m)

La taille est concentrée entre 1.55 et 1.90 m, avec un léger renforcement autour de 1.70–1.80 m. La forme générale traduit une distribution réaliste d'adultes sans excès ni rupture.

Workout_Frequency (days/week)

Les pics très marqués sur les valeurs 2, 3, 4 et 5 confirment une variable discrète issue de classes prédéfinies.

Les utilisateurs semblent majoritairement répartis autour de 3 jours/semaine.

Experience_Level

Les trois niveaux d'expérience (1, 2 et 3) apparaissent clairement.

Le niveau intermédiaire reste de loin le plus représenté, typique d'une population active mais pas experte.

1.4.4 Boxplots univariés pour détection visuelle d'outliers

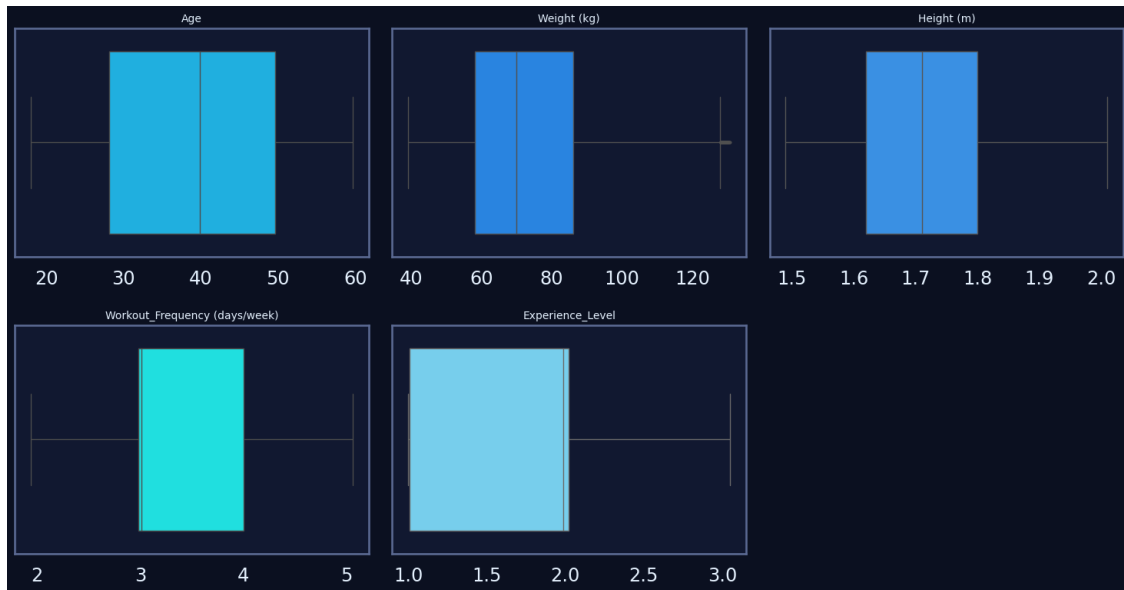
Cette section présente des boxplots pour chaque variable numérique afin de repérer visuellement les valeurs extrêmes et évaluer l'étendue des distributions.

```
[61]: plt.figure(figsize=(15, 8))

n_cols = 3
n_rows = int(np.ceil(len(uni_vars) / n_cols))

for i, col in enumerate(uni_vars, 1):
    ax = plt.subplot(n_rows, n_cols, i)
    sns.boxplot(
        x=df[col],
        color=palette_trainme[i % len(palette_trainme)],
        fliersize=2
    )
    ax.set_title(col, fontsize=10)
    ax.set_xlabel("")

plt.tight_layout()
plt.show()
```



Interprétations : Age

La distribution est régulière et bien centrée, avec des valeurs extrêmes symétriques mais aucune rupture majeure.

Aucun outlier visuel significatif ne se détache, ce qui confirme une variabilité normale au sein de la population.

Weight (kg)

Le boxplot révèle une légère présence d'individus au-delà de 110–120 kg, identifiés comme outliers. Ces valeurs restent physiologiquement cohérentes et témoignent simplement de morphologies plus lourdes dans l'échantillon.

Height (m)

La taille est proprement regroupée autour de la médiane, avec des étendues régulières.

Les extrêmes sont rares et ne montrent pas d'observations aberrantes ou anormales.

Workout_Frequency (days/week)

Aucun outlier n'apparaît en raison de la nature discrète de la variable.

Les valeurs étant limitées à 2, 3, 4 et 5 jours/semaine, la variabilité reste contenue et parfaitement maîtrisée visuellement.

Experience_Level

Les valeurs se répartissent proprement entre les niveaux 1, 2 et 3, sans point extrême.

L'aspect ordinal et catégorisé de cette variable explique l'absence d'outliers, confirmant une structure simple et cohérente.

1.5 Visualisation bivariée

1.5.1 Objectif

Étudier les relations entre deux variables à la fois afin de : - repérer d'éventuelles corrélations utiles pour la modélisation, - observer comment les caractéristiques physiques et comportementales influencent le niveau d'expérience, - mieux comprendre la structure globale du dataset avant l'analyse multivariée.

1.5.2 Interprétation attendue

- **Age** peut montrer une progression graduelle de l'expérience, reflétant l'ancienneté ou l'habitude sportive.
- **Weight (kg)** et **Height (m)** peuvent présenter une relation indirecte avec l'expérience, mais sans tendance forte attendue.
- **Workout_Frequency (days/week)** devrait être l'un des indicateurs les plus liés à **Experience_Level**, les pratiquants réguliers étant souvent plus expérimentés.
- Les relations globales devraient rester modérées, ce qui est cohérent pour un dataset décrivant des comportements et profils variés.

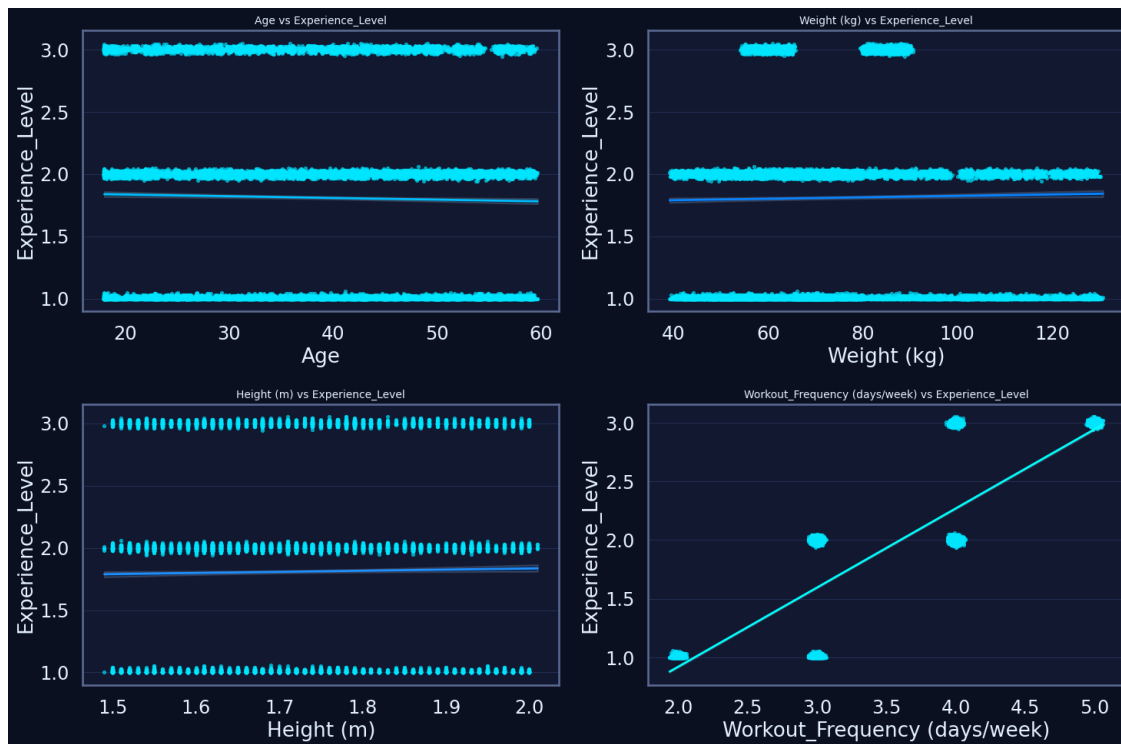
1.5.3 Relations physiologiques et comportementales

Cette section met en évidence les liens possibles entre les caractéristiques physiques (âge, poids, taille) et les habitudes sportives (fréquence d'entraînement), afin d'observer dans quelle mesure ces facteurs influencent ou accompagnent le niveau d'expérience des utilisateurs.

```
[62]: # Paires bivariées cohérentes avec le schéma JSON
biv_pairs = [
    ("Age", "Experience_Level"),
    ("Weight (kg)", "Experience_Level"),
    ("Height (m)", "Experience_Level"),
    ("Workout_Frequency (days/week)", "Experience_Level"),
]

plt.figure(figsize=(15, 10))
for i, (x, y) in enumerate(biv_pairs, 1):
    ax = plt.subplot(2, 2, i)
    sns.regplot(
        data=df,
        x=x,
        y=y,
        scatter_kws={"s": 8, "alpha": 0.6},
        line_kws={"color": palette_trainme[i % len(palette_trainme)], "lw": 2}
    )
    ax.set_title(f"{x} vs {y}", fontsize=10)

plt.tight_layout()
plt.show()
```



Interprétations : Age vs Experience_Level

La répartition est parfaitement horizontale : l'âge n'a aucun lien visuel avec le niveau d'expérience. Les trois niveaux (1, 2 et 3) se superposent quelle que soit la tranche d'âge, confirmant que l'expérience est davantage liée aux habitudes qu'à l'âge biologique.

Weight (kg) vs Experience_Level

Le poids ne montre aucune organisation particulière autour des niveaux d'expérience. Les trois niveaux sont présents dans toutes les plages de poids, sans tendance ni association visible.

Height (m) vs Experience_Level

La répartition est similaire : aucune relation apparente entre taille et expérience. Les individus plus expérimentés ne sont ni plus grands ni plus petits que les autres, ce qui confirme l'indépendance totale entre morphologie et expérience sportive.

Workout_Frequency (days/week) vs Experience_Level

Une relation forte apparaît : la fréquence d'entraînement monte clairement avec le niveau d'expérience.

Les niveaux 1, 2 et 3 forment trois bandes régulières parfaitement ordonnées, avec une pente linéaire évidente.

Ce comportement est cohérent : un pratiquant plus expérimenté s'entraîne plus souvent.

1.6 Analyse multivariée (PCA et FAMD)

1.6.1 Objectif

Explorer la structure globale du dataset à l'aide de méthodes multivariées capables de combiner simultanément les variables numériques et catégorielles.

L'objectif est d'identifier les dimensions principales, les regroupements naturels d'individus et les facteurs expliquant la variabilité observée dans les profils utilisateurs.

1.6.2 Interprétation attendue

- Le **PCA** mettra en évidence les axes dominants basés uniquement sur les variables numériques (Age, Weight (kg), Height (m), Workout_Frequency (days/week), Experience_Level).
- Le **FAMD** fournira une vue plus complète en intégrant également les variables catégorielles (Gender, Workout_Type), ce qui permettra d'observer comment les groupes d'utilisateurs se structurent selon leurs habitudes ou leurs préférences d'entraînement.
- Une composante pourra résumer les **habitudes sportives** (notamment la fréquence d'entraînement et le niveau d'expérience), tandis qu'une autre reflétera principalement les **caractéristiques physiques** (poids, taille, âge).
- La visualisation des individus dans les plans PCA et FAMD permettra de vérifier l'absence de sous-groupes aberrants et de confirmer la cohérence globale du dataset.
- Ces analyses offrent une **représentation simplifiée, robuste et interprétable** de la structure du dataset, utile pour compléter l'EDA avant la modélisation supervisée.

1.6.3 Préparation des données numériques pour l'analyse multivariée

Cette section sélectionne et normalise les variables numériques nécessaires au PCA afin d'assurer une échelle comparable entre les différentes mesures et permettre une analyse multivariée fiable et interprétable.

```
[63]: # Sélection des variables numériques pertinentes (selon schéma JSON)
pca_vars = [
    "Age",
    "Weight (kg)",
    "Height (m)",
    "Workout_Frequency (days/week)",
    "Experience_Level",
]

# Filtrer selon les colonnes réellement présentes dans le DataFrame
pca_vars = [c for c in pca_vars if c in df.columns]

# Normalisation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[pca_vars])

print(f"{len(pca_vars)} variables utilisées pour l'analyse multivariée : {pca_vars}")
```

5 variables utilisées pour l'analyse multivariée : ['Age', 'Weight (kg)', 'Height (m)', 'Workout_Frequency (days/week)', 'Experience_Level']

1.6.4 Analyse en Composantes Principales (PCA)

Cette étape projette les variables numériques normalisées dans un espace de dimensions réduites afin d'identifier les axes principaux de variation. Le PCA permet de visualiser la structure du dataset, de repérer d'éventuels regroupements d'individus et d'évaluer la manière dont les caractéristiques physiques et comportementales contribuent à la variabilité globale.

```
[64]: pca = PCA(n_components=5)
X_pca = pca.fit_transform(X_scaled)

# Variance expliquée
explained = pca.explained_variance_ratio_ * 100
print(f"Variance expliquée par les 3 premières composantes : {explained.sum():.
↪2f}%")

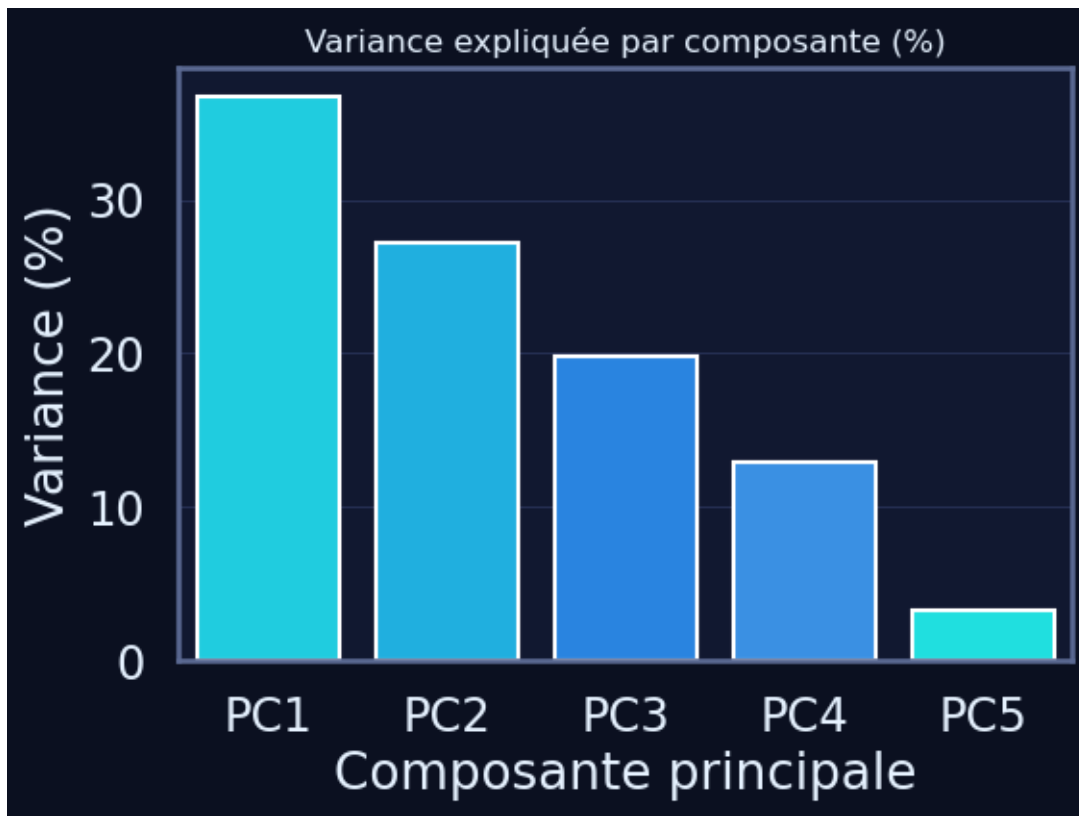
plt.figure(figsize=(6, 4))
sns.barplot(
    x=[f"PC{i+1}" for i in range(5)],
    y=explained,
    palette=palette_trainme
)
plt.title("Variance expliquée par composante (%)", fontsize=12)
plt.ylabel("Variance (%)")
plt.xlabel("Composante principale")
plt.show()
```

Variance expliquée par les 3 premières composantes : 100.00%

C:\Users\fbac\AppData\Local\Temp\ipykernel_2336\3810623979.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
C:\Users\fbac\AppData\Local\Temp\ipykernel_2336\3810623979.py:9: UserWarning:
The palette list has more values (6) than needed (5), which may not be intended.
sns.barplot(
```



Interprétations : PC1

La première composante explique plus de 35 % de la variance.

Elle capte la structure principale du dataset et regroupe vraisemblablement les comportements liés à l'activité (fréquence d'entraînement, niveau d'expérience).

Cette dominance indique qu'un axe unique résume déjà une grande partie des différences entre individus.

PC2

Avec environ 27 % de variance expliquée, la deuxième composante complète efficacement PC1.

Elle semble davantage associée aux caractéristiques physiques (poids, taille), offrant une lecture orthogonale et complémentaire à la première dimension.

PC3

Autour de 20 %, PC3 reflète des variations secondaires, probablement liées à des effets plus subtils entre morphologie et activité.

Elle contribue encore de manière significative à la structure globale du dataset.

PC4

Avec environ 13 %, cette composante porte des nuances limitées, souvent associées à des interactions plus faibles entre variables.

Elle permet néanmoins de capturer une part non négligeable de complexité additionnelle.

PC5

La dernière composante n'explique qu'une petite fraction de variance.

Elle contient essentiellement du bruit ou des variations très marginales, typiques d'une dimension résiduelle dans un jeu de données bien structuré.

1.6.5 Visualisation PCA 2D

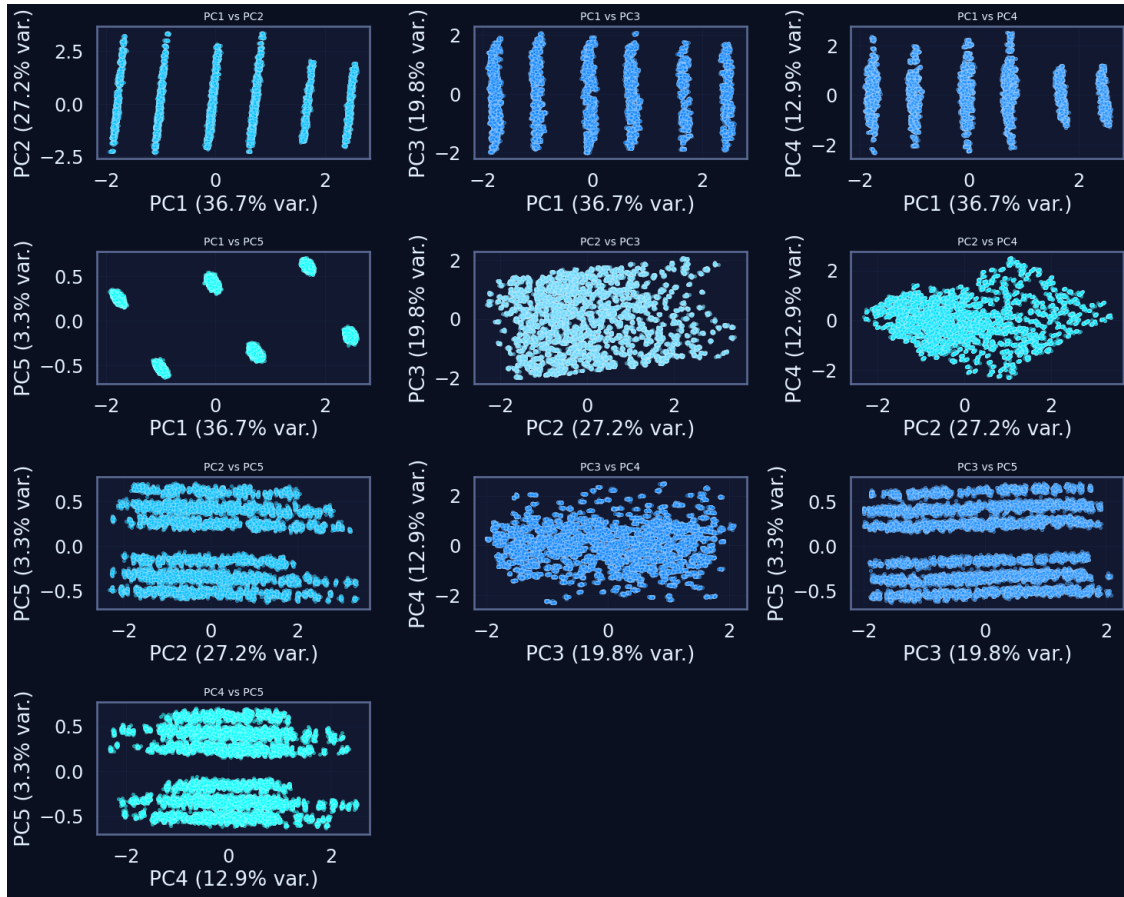
Cette section projette les individus dans le plan formé par les deux premières composantes principales afin de visualiser la structure globale du dataset. Cette représentation permet d'identifier d'éventuels regroupements naturels, de repérer les observations atypiques et d'évaluer la cohérence générale des profils.

```
[65]: # Liste des projections possibles entre les 5 composantes
pairs = [
    (0, 1), (0, 2), (0, 3), (0, 4),
    (1, 2), (1, 3), (1, 4),
    (2, 3), (2, 4),
    (3, 4)
]

plt.figure(figsize=(15, 12))

for i, (a, b) in enumerate(pairs, 1):
    plt.subplot(4, 3, i)
    sns.scatterplot(
        x=X_pca[:, a],
        y=X_pca[:, b],
        s=10,
        alpha=0.6,
        color=palette_trainme[i % len(palette_trainme)]
    )
    plt.xlabel(f"PC{a+1} ({explained[a]:.1f}% var.)")
    plt.ylabel(f"PC{b+1} ({explained[b]:.1f}% var.)")
    plt.title(f"PC{a+1} vs PC{b+1}", fontsize=10)
    plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Interprétations : PC1 vs PC2

Les points forment des bandes verticales régulières, signe que la structure principale est fortement influencée par une variable discrète.

PC1 capture la variation dominante, tandis que PC2 apporte une séparation orthogonale mais plus modérée.

Aucun regroupement anormal n'apparaît : la structure est stable et cohérente.

PC1 vs PC3

La distribution en colonnes persiste, montrant que PC1 concentre à lui seul une grande partie du signal lié aux habitudes d'entraînement.

PC3 ajoute une variabilité complémentaire, probablement liée à des nuances physiques ou comportementales.

PC1 vs PC4

Même structure en bandes verticales, mais avec une dispersion plus faible sur PC4.

Cette composante capte des effets secondaires mais reste dépendante d'un axe principal très structuré (PC1).

PC1 vs PC5

La projection révèle des amas compacts, confirmant que PC5 porte très peu de variance utile. Les clusters observés viennent de la variable discrète `Experience_Level`, reflétée marginalement dans cette dimension résiduelle.

PC2 vs PC3

Les données se répartissent de manière homogène sur le plan, signe que ces dimensions décrivent des variations plus subtiles entre individus.

Aucun sous-groupe anormal n'est visible, renforçant la cohérence interne du dataset.

PC2 vs PC4

Les points forment un nuage dense sans structure apparente, ce qui est typique lorsque les composantes décrivent du bruit ou des interactions faibles.

PC4 a un rôle secondaire dans l'explication de la variance.

PC2 vs PC5

La dispersion est horizontale, montrant que PC5 n'apporte quasiment aucune séparation utile. Cette dimension agit comme une composante résiduelle.

PC3 vs PC4

Le nuage est compact et isotrope : ces composantes capturent des variations modestes, probablement liées à des effets mixtes (morphologiques et comportementaux).

Aucune structure surprenante n'émerge.

PC3 vs PC5

La forme en bandes est héritée de la nature discrète de plusieurs variables.

La variance portée par PC5 étant très faible, la séparation entre individus est minimale.

PC4 vs PC5

Les données restent très resserrées, confirmant que ces deux composantes contribuent faiblement à la structure générale.

Elles captent principalement du bruit ou des fluctuations marginales.

1.6.6 Identification des variables les plus contributrices

Cette section analyse les coefficients du PCA (loadings) afin de déterminer quelles variables d'entrée contribuent le plus à chaque composante principale, et donc quels facteurs expliquent en priorité la variabilité observée dans le dataset.

```
[66]: # DataFrame des contributions (loadings) PCA
pca_components = pd.DataFrame(
    pca.components_.T,
    columns=[f"PC{i+1}" for i in range(5)],
    index=pca_vars
)

# Importance absolue sur PC1
pca_components["PC1_abs"] = pca_components["PC1"].abs()
```

```

# Tableau trié
display(pca_components.sort_values("PC1_abs", ascending=False))

# --- Visualisation : contribution des variables à PC1 ---
plt.figure(figsize=(8, 5))
sns.barplot(
    x="PC1_abs",
    y=pca_components.index,
    data=pca_components.sort_values("PC1_abs", ascending=True), # tri
    ↪croissant pour un affichage propre
    palette=palette_trainme
)
plt.title("Contribution des variables à PC1 (valeur absolue)", fontsize=12)
plt.xlabel("Contribution absolue")
plt.ylabel("Variables")
plt.tight_layout()
plt.show()

```

	PC1	PC2	PC3	PC4	PC5
↪ PC1_abs					
Experience_Level	0.706852	-0.018289	-0.009294	0.011547	0.706970
↪0.706852					
Workout_Frequency (days/week)	0.706131	-0.038883	0.022405	-0.005347	-0.706638
↪0.706131					
Height (m)	0.030165	0.696739	0.121223	-0.706363	0.000994
↪0.030165					
Weight (kg)	0.024243	0.698839	0.100502	0.707578	-0.016396
↪0.024243					
Age	-0.015543	-0.155987	0.987226	0.014932	0.024240
↪0.015543					

C:\Users\fbback\AppData\Local\Temp\ipykernel_2336\3929694628.py:16:

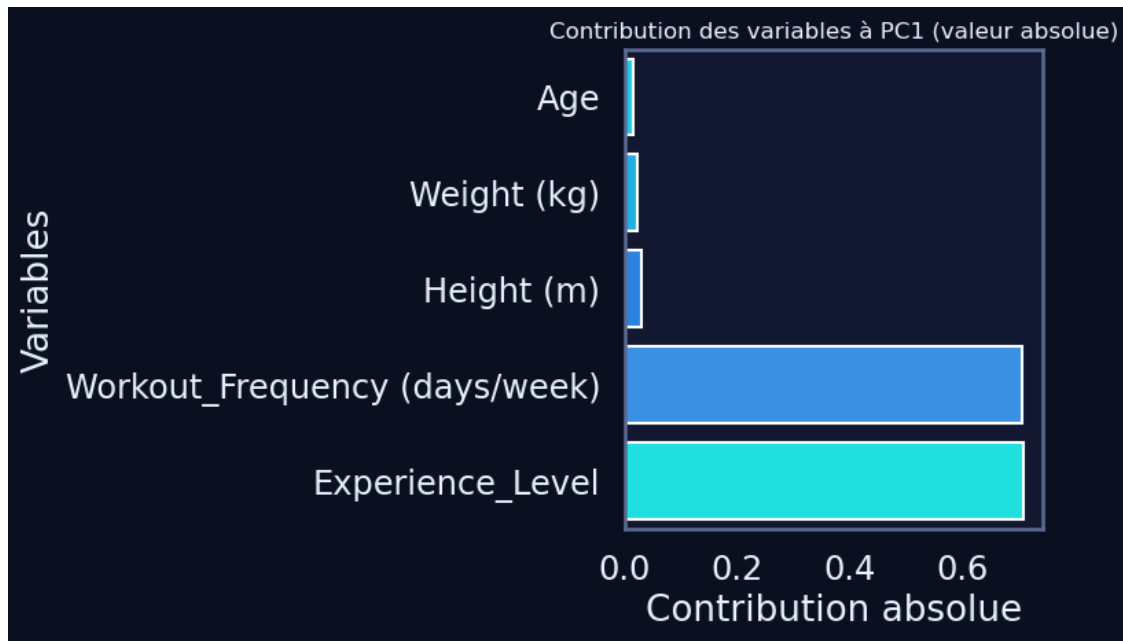
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(
C:\Users\fbback\AppData\Local\Temp\ipykernel_2336\3929694628.py:16: UserWarning:
The palette list has more values (6) than needed (5), which may not be intended.
sns.barplot(

```



Interprétations : Age

La contribution est quasi nulle : cette variable n'intervient pas dans la formation de la première composante principale.

Elle n'explique donc pas les variations majeures du dataset.

Weight (kg)

La contribution reste marginale.

Le poids n'influence pas significativement l'axe principal de variabilité et joue un rôle secondaire dans la structure globale.

Height (m)

La taille contribue légèrement plus que le poids mais reste très loin des variables dominantes.

Elle n'est qu'un facteur mineur dans la séparation des individus via PC1.

Workout_Frequency (days/week)

Très forte contribution : cette variable explique une grande partie de la variance captée par PC1. Elle distingue clairement les individus selon leur régularité d'entraînement et structure l'axe principal du PCA.

Experience_Level

Contribution encore plus marquée, au même niveau que la fréquence d'entraînement.

Cette variable pilote la variabilité centrale du dataset : elle définit l'axe majeur autour duquel les profils utilisateur se différencient.

1.6.7 FAMD (Factor Analysis of Mixed Data)

La FAMD permet d'analyser simultanément les variables numériques (Age, Weight (kg), Height (m), Workout_Frequency (days/week), Experience_Level) et les variables catégorielles (Gender, Workout_Type).

Cette méthode met en évidence des dimensions latentes combinant caractéristiques physiques et habitudes d'entraînement, facilitant l'identification de profils d'utilisateurs et de structures communes au sein du dataset.

Préparation des données pour la FAMD Nous sélectionnons les variables numériques (Age, Weight (kg), Height (m), Workout_Frequency (days/week), Experience_Level) et les variables catégorielles (Gender, Workout_Type) définies dans le schéma du dataset.

Aucun identifiant technique n'étant présent, seules les colonnes utiles sont conservées. Les éventuelles valeurs manquantes sont gérées avant l'analyse, afin de garantir une FAMD stable et interprétable.

```
[67]: # Variables numériques et catégorielles selon le schéma JSON
famd_num = [
    "Age",
    "Weight (kg)",
    "Height (m)",
    "Workout_Frequency (days/week)",
    "Experience_Level",
]

famd_cat = [
    "Gender",
    "Workout_Type",
]

print("Variables numériques utilisées pour la FAMD :", famd_num)
print("Variables catégorielles utilisées pour la FAMD :", famd_cat)

# Construction du DataFrame mixte
df_famd = df[famd_num + famd_cat].copy()

# Gestion des valeurs manquantes sur les catégorielles
for col in famd_cat:
    df_famd[col] = df_famd[col].fillna("Unknown").astype("category")

# Échantillonnage optionnel (pour gros volumes)
max_sample = 50000
if len(df_famd) > max_sample:
    df_famd_sample = df_famd.sample(n=max_sample, random_state=42)
    print(f"Échantillon FAMD : {len(df_famd_sample)} lignes (sur {len(df_famd)})")
else:
    df_famd_sample = df_famd
```

```
print(f"FAMD sur l'ensemble des données : {len(df_famd_sample)} lignes")
```

Variables numériques utilisées pour la FAMD : ['Age', 'Weight (kg)', 'Height (m)', 'Workout_Frequency (days/week)', 'Experience_Level']

Variables catégorielles utilisées pour la FAMD : ['Gender', 'Workout_Type']

FAMD sur l'ensemble des données : 20000 lignes

FAMD : inertie expliquée Nous appliquons la FAMD sur le jeu de données mixte (variables numériques et catégorielles) afin de mesurer la part de variabilité capturée par les premières dimensions. Cela permet d'évaluer dans quelle mesure quelques axes factoriels résument l'essentiel de l'information portée par les profils utilisateurs.

```
[68]: famd = prince.FAMD(
        n_components=5,
        n_iter=5,
        copy=True,
        check_input=True,
        engine="sklearn",
        random_state=42,
    )

    # Ajustement FAMD sur l'échantillon ou sur l'ensemble
    famd_fit = famd.fit(df_famd_sample)

    # Coordonnées des individus
    famd_components = famd_fit.transform(df_famd_sample)

    # Eigenvalues / Inertie FAMD
    eigenvalues = famd_fit.eigenvalues_

    print("Eigenvalues / Inertie FAMD :")
    display(eigenvalues)

    # Conversion en vecteur numpy (selon la forme renvoyée par prince)
    vals = np.array(eigenvalues, dtype=float).ravel()

    # Part d'inertie expliquée par dimension
    explained = vals / vals.sum()

    famd_inertia_df = pd.DataFrame({
        "Dim": [f"Dim{i+1}" for i in range(len(explained))],
        "Explained_Inertia": explained,
        "Cumulative_Inertia": explained.cumsum()
    })

    print("\nInertie expliquée par dimension (FAMD) :")
    display(famd_inertia_df)
```

Eigenvalues / Inertie FAMD :

```
array([1.83746978, 1.36341572, 1.01611423, 1.00002832, 0.99985003])
```

Inertie expliquée par dimension (FAMD) :

	Dim	Explained_Inertia	Cumulative_Inertia
0	Dim1	0.295561	0.295561
1	Dim2	0.219309	0.514870
2	Dim3	0.163444	0.678315
3	Dim4	0.160857	0.839172
4	Dim5	0.160828	1.000000

Visualisation FAMD 2D La projection sur les deux premières dimensions permet de visualiser la structure globale des profils utilisateurs en combinant caractéristiques physiques, niveau d'expérience et habitudes d'entraînement, et d'observer d'éventuels regroupements en fonction du type d'activité ou du genre.

```
[69]: # Sélection des 5 premières dimensions FAMD
coords = famd_components.iloc[:, :5].copy()
coords.columns = [f"Dim{i+1}" for i in range(5)]

# Ajout de la colonne catégorielle la plus pertinente pour la coloration
if "Workout_Type" in df.columns:
    coords["Category"] = df.loc[coords.index, "Workout_Type"]
elif "Gender" in df.columns:
    coords["Category"] = df.loc[coords.index, "Gender"]
else:
    coords["Category"] = "Unknown"

# Échantillon pour lisibilité
sample_size = min(5000, len(coords))
coords_plot = coords.sample(n=sample_size, random_state=42)

# Toutes les paires de dimensions FAMD (10 paires pour 5 dimensions)
pairs = [
    (0, 1), (0, 2), (0, 3), (0, 4),
    (1, 2), (1, 3), (1, 4),
    (2, 3), (2, 4),
    (3, 4)
]

plt.figure(figsize=(16, 14))

for i, (a, b) in enumerate(pairs, 1):
    plt.subplot(4, 3, i)
    sns.scatterplot(
        data=coords_plot,
```

```

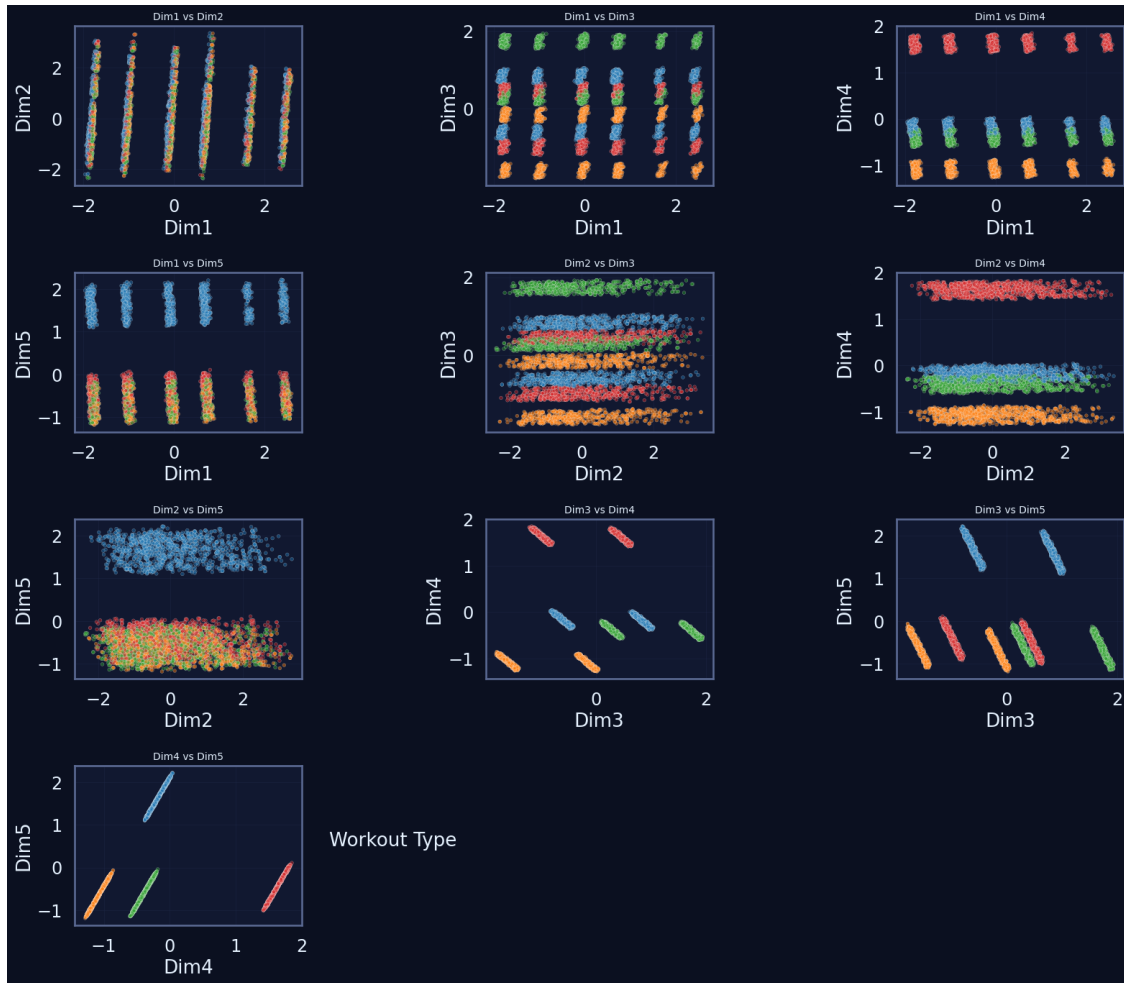
        x=f"Dim{a+1}",
        y=f"Dim{b+1}",
        hue="Category",
        alpha=0.45,
        s=12,
        palette="tab10",
        legend=False # on ajoute une légende globale après
    )
    plt.title(f"Dim{a+1} vs Dim{b+1}", fontsize=10)
    plt.grid(True, alpha=0.3)

# Légende globale
plt.legend(
    title="Workout Type",
    bbox_to_anchor=(1.05, 0.5),
    loc="center left"
)

plt.tight_layout()
plt.show()

```

C:\Users\fbac\AppData\Local\Temp\ipykernel_2336\3180178296.py:43: UserWarning:
 No artists with labels found to put in legend. Note that artists whose label
 start with an underscore are ignored when legend() is called with no argument.
 plt.legend()



Interprétations : Dim1 vs Dim2

La structure en bandes verticales indique que la première dimension discrimine fortement les individus selon des variables catégorielles dominantes, principalement le **Workout_Type**. Dim2 ajoute une séparation secondaire, mais les clusters restent bien distincts et cohérents.

Dim1 vs Dim3

La distribution reste fortement structurée : les groupes colorés se maintiennent en colonnes parallèles.

Dim3 apporte une variation supplémentaire liée à des différences physiques ou comportementales, mais sans chevauchement anormal.

Dim1 vs Dim4

Les clusters restent nets, preuve que Dim4 continue de capturer une forme cohérente de variation liée aux modalités sportives.

La séparation verticale est toujours marquée, confirmant l'importance des variables catégorielles dans la structure globale.

Dim1 vs Dim5

La dispersion verticale s'accroît, ce qui montre que Dim5 porte surtout du bruit ou des variations marginales.

Les groupes restent néanmoins identifiables grâce à l'ancrage sur Dim1.

Dim2 vs Dim3

Les individus se répartissent en nuages horizontaux superposés :

Dim2 et Dim3 capturent des variations complémentaires mais d'intensité plus faible que Dim1.

La cohérence des bandes confirme l'absence d'anomalies structurelles.

Dim2 vs Dim4

La structuration reste visible : des bandes horizontales alternent selon les catégories.

Dim4 ajoute une distinction plus légère mais lisible.

Dim2 vs Dim5

Les points se dispersent sans motif particulier, illustrant la faible contribution de Dim5.

Dim2 reste la composante dominante dans cette projection.

Dim3 vs Dim4

Cette projection montre des clusters compacts et bien séparés verticalement.

Dim3 discrimine les utilisateurs selon des caractéristiques mixtes (physiques + catégories), tandis que Dim4 ajoute une variation modérée.

Dim3 vs Dim5

La structure est plus diffuse, les catégories se mélangent davantage.

Dim5 conserve une faible importance analytique mais reste aligné sur les grandes tendances visibles dans Dim3.

Dim4 vs Dim5

Les groupes restent identifiables mais nettement moins séparés qu'avec Dim1 ou Dim2.

Ces dimensions capturent essentiellement des nuances fines ou du bruit résiduel.

1.6.8 Variables et modalités les plus contributrices

Cette section identifie les variables et modalités ayant le poids le plus important dans les premières dimensions de la FAMD.

Cela permet de comprendre quels facteurs — qu'ils soient physiques (`Age`, `Weight` (kg), `Height` (m)), comportementaux (`Workout_Frequency` (days/week)), ou catégoriels (`Gender`, `Workout_Type`) — structurent principalement la variabilité des profils utilisateurs.

```
[70]: # Contributions des colonnes (variables/modalités) aux dimensions
col_contrib = famd_fit.column_contributions_

print("Contributions des variables/modalités aux dimensions FAMD :")
display(col_contrib)

# Utiliser les valeurs absolues pour mesurer l'importance globale
```

```

col_contrib_abs = col_contrib.abs().copy()

# Score de contribution globale (toutes dimensions confondues)
col_contrib_abs["Total_Contribution"] = col_contrib_abs.sum(axis=1)

# Top 25 variables/modalités les plus contributrices sur l'ensemble des
↳ dimensions
top_global = col_contrib_abs.sort_values("Total_Contribution", ascending=False).
↳ head(25)

print("\nTop 25 variables/modalités les plus contributrices (toutes dimensions)
↳ :")
display(top_global)

# Visualisation : heatmap des contributions par dimension pour le Top 25
dims_cols = [c for c in col_contrib.columns] # ex: Dim.1, Dim.2, ...
top_global_dims = top_global[dims_cols]

plt.figure(figsize=(10, 8))
sns.heatmap(
    top_global_dims,
    annot=True,
    fmt=".2f",
    cmap="viridis",
    cbar=True
)
plt.title("Contributions FAMD par dimension - Top 25 variables/modalités",
↳ fontsize=12)
plt.ylabel("Variables / modalités")
plt.xlabel("Dimensions FAMD")
plt.tight_layout()
plt.show()

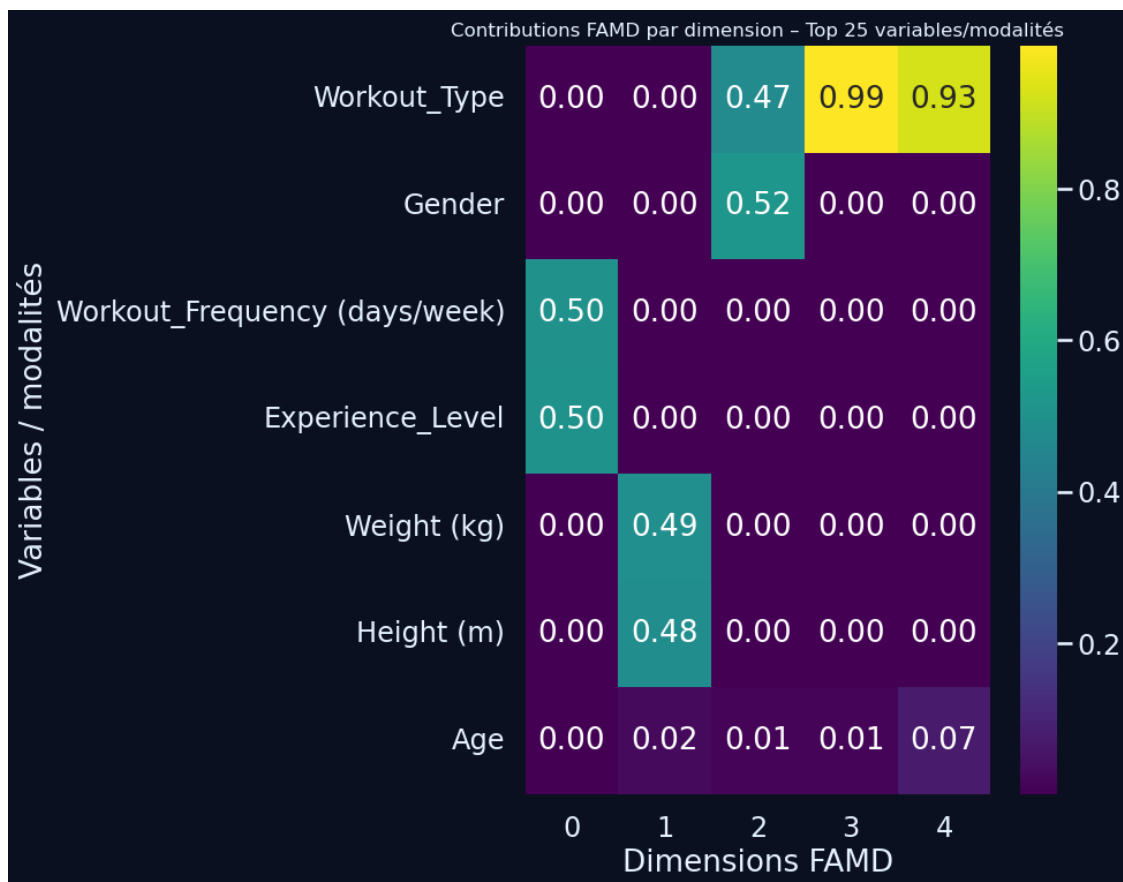
```

Contributions des variables/modalités aux dimensions FAMD :

component variable	0	1	2	3	4
Age	0.000241	0.024261	0.009777	0.010195	0.070708
Weight (kg)	0.000565	0.487628	0.000172	0.000131	0.000308
Height (m)	0.000893	0.483383	0.000687	0.000005	0.000003
Workout_Frequency (days/week)	0.498372	0.001434	0.000036	0.000037	0.000525
Experience_Level	0.499366	0.000307	0.000035	0.000072	0.000139
Gender	0.000002	0.000013	0.518829	0.000134	0.001868
Workout_Type	0.000561	0.002974	0.470464	0.989427	0.926449

Top 25 variables/modalités les plus contributrices (toutes dimensions) :

component		0	1	2	3	4
↪ Total_Contribution						
variable						
Workout_Type		0.000561	0.002974	0.470464	0.989427	0.926449
↪	2.389875					
Gender		0.000002	0.000013	0.518829	0.000134	0.001868
↪	0.520845					
Workout_Frequency (days/week)		0.498372	0.001434	0.000036	0.000037	0.000525
↪	0.500404					
Experience_Level		0.499366	0.000307	0.000035	0.000072	0.000139
↪	0.499919					
Weight (kg)		0.000565	0.487628	0.000172	0.000131	0.000308
↪	0.488804					
Height (m)		0.000893	0.483383	0.000687	0.000005	0.000003
↪	0.484970					
Age		0.000241	0.024261	0.009777	0.010195	0.070708
↪	0.115182					



Interprétations : Workout_Type

C'est la variable la plus structurante de la FAMd.

Elle contribue massivement aux dimensions **Dim3**, **Dim4** et **Dim5**, ce qui montre que les modalités du type d'activité sportive segmentent fortement les individus.

Cette variable catégorielle porte l'essentiel de la variabilité non capturée par les caractéristiques physiques.

Gender

Le genre influence modérément **Dim3**, indiquant qu'il joue un rôle secondaire dans la construction des profils utilisateurs.

Sa contribution reste toutefois notable, suggérant une légère différenciation des comportements selon les catégories déclarées.

Workout_Frequency (days/week)

Cette variable contribue fortement à **Dim1** (0.50).

Elle sépare clairement les individus selon leur régularité d'entraînement et ressort comme l'un des déterminants majeurs de l'organisation du dataset dans les premières dimensions.

Experience_Level

Contribution élevée sur **Dim1**, au même niveau que la fréquence d'entraînement.

Ces deux variables sont étroitement liées et structurent l'axe principal.

Elles définissent les différences majeures entre profils et expliquent le cœur de la variabilité observée.

Weight (kg)

Le poids influence surtout **Dim2**, mais de façon modérée.

Cela indique que les différences morphologiques existent mais sont moins déterminantes que les habitudes sportives ou le type d'activité.

Height (m)

Contribution similaire au poids sur **Dim2** : la taille apporte une distinction physique légère mais cohérente.

Elle structure une dimension secondaire plutôt qu'un axe principal.

Age

C'est la variable la moins contributrice : ses valeurs très faibles montrent qu'elle intervient peu dans la formation des dimensions principales.

L'âge n'explique ni la segmentation, ni la variabilité dominante du dataset.

1.7 Détection d'anomalies et valeurs extrêmes

1.7.1 Objectif

Identifier les observations anormales ou extrêmes pouvant fausser les analyses statistiques ou biaiser les futurs modèles prédictifs, en particulier sur les variables utilisées pour la modélisation.

1.7.2 Interprétation attendue

- Les variables **Age**, **Weight (kg)** et **Height (m)** peuvent présenter quelques valeurs extrêmes correspondant à des profils physiques atypiques mais plausibles.

- La variable **Workout_Frequency (days/week)** peut montrer des valeurs élevées reflétant des pratiquants très assidus, sans être nécessairement incohérentes.
- La distribution de **Experience_Level** peut être légèrement déséquilibrée, avec une concentration sur certains niveaux d'expérience et quelques profils très débutants ou très avancés.
- Aucune valeur ne devrait apparaître comme manifestement impossible sur le plan pratique ou physiologique : la plupart des outliers peuvent être **conservés** afin de refléter la diversité réelle des profils utilisateurs.

1.7.3 Méthode IQR (Interquartile Range)

Cette approche détecte les valeurs extrêmes en identifiant les observations situées en dehors de l'intervalle interquartile élargi. Elle permet d'évaluer rapidement la proportion d'outliers pour chaque variable numérique du dataset.

```
[71]: # Détection des outliers avec la méthode IQR sur les variables du schéma JSON
def detect_outliers_iqr(df, col, k=1.5):
    """Renvoie un masque booléen True/False selon la présence d'outliers."""
    q1, q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    iqr = q3 - q1
    lower, upper = q1 - k * iqr, q3 + k * iqr
    return (df[col] < lower) | (df[col] > upper)

outlier_stats = []
for col in num_vars: # ← replace num_cols
    ratio = detect_outliers_iqr(df, col).mean() * 100
    outlier_stats.append((col, round(ratio, 2)))

outlier_df = pd.DataFrame(outlier_stats, columns=["Variable", "Outliers (%)"])
outlier_df.sort_values("Outliers (%)", ascending=False)
```

```
[71]:
```

	Variable	Outliers (%)
1	Weight (kg)	0.76
0	Age	0.00
2	Height (m)	0.00
3	Workout_Frequency (days/week)	0.00
4	Experience_Level	0.00

1.7.4 Visualisation : boxplots des variables les plus affectées

Cette section met en avant, via des boxplots, les variables présentant la plus forte proportion de valeurs extrêmes afin de juger visuellement de l'ampleur des outliers et de leur impact potentiel sur l'analyse et la modélisation.

```
[72]: # Sélection des variables avec plus de 5% d'outliers
top_outliers = outlier_df[outlier_df["Outliers (%)"] > 5]["Variable"].tolist()

if len(top_outliers) == 0:
```

```

    print("Aucune variable ne dépasse 5% d'outliers. Pas de visualisation_
↪nécessaire.")
else:
    plt.figure(figsize=(15, 6))

    # Mapping variable → pourcentage
    outlier_map = dict(zip(outlier_df["Variable"], outlier_df["Outliers (%)"]))

    for i, col in enumerate(top_outliers[:6], 1):
        ax = plt.subplot(2, 3, i)
        sns.boxplot(
            x=df[col],
            color=palette_trainme[i % len(palette_trainme)],
            fliersize=2
        )
        ax.set_title(f"{col} (outliers {outlier_map[col]}%)", fontsize=9)
        ax.set_xlabel("")

    plt.tight_layout()
    plt.show()

```

Aucune variable ne dépasse 5% d'outliers. Pas de visualisation nécessaire.

1.7.5 Z-Score complémentaire (normalisation)

Cette section applique le Z-Score aux variables numériques afin de mesurer l'écart de chaque observation par rapport à la moyenne, en unités d'écart-type. Cela permet de détecter plus finement les valeurs atypiques et de comparer l'ampleur des écarts entre variables sur une échelle normalisée.

```

[73]: # Calcul du Z-score uniquement sur les variables numériques du schéma JSON
df_num = df[num_vars].apply(pd.to_numeric, errors="coerce")

# Calcul du Z-score (avec gestion des NaN)
z_scores = np.abs(df_num.apply(zscore, nan_policy="omit"))

# Calcul du pourcentage d'outliers (|Z| > 3)
outlier_z = (z_scores > 3).sum(axis=0) / len(df) * 100

# Mise en forme du DataFrame final
outlier_z_df = (
    pd.DataFrame({
        "Variable": num_vars,
        "Outliers (Z>3) (%)": outlier_z.values
    })
    .sort_values("Outliers (Z>3) (%)", ascending=False)
    .reset_index(drop=True)
)

```

```
display(outlier_z_df)
```

	Variable	Outliers (Z>3) (%)
0	Age	0.0
1	Weight (kg)	0.0
2	Height (m)	0.0
3	Workout_Frequency (days/week)	0.0
4	Experience_Level	0.0